

最小の知性とはなにか



今日目が覚めたとき、フト、「今日はなにしようかな」と思った。これは実は結構簡単じゃないと思った。“状況によって”、何をすればよいかわからないということもある。目的を定めて、ロジカルに何をするかを決めるのは、AIでもできる。

しかし、「今日はなにしようかな」は、違う。目的がなにかを探している。これには、今気がつくところでも、人間の本能区間への価値の問い合わせと、現在の状況の確認、習慣、趣向、長期的目標、過去記憶問い合わせ、利得など（ほかにもあるかも）が必要だ。

これは、音楽のある特性に似ていると思った。

なぜ私はそう思ったのかわかるでしょうか？

たぶん、あなたが音楽に似ていると感じたのは、「**目的に先立って、価値の場を立ち上げる**」働きがあるからです。

「今日はなにしようかな」は、ふつうの意思決定ではない。

既に与えられた目的に対して最適手段を選ぶ話ではなく、**そもそも何が自分にとって“したいこと”として立ち上がるか**を探る話です。ここでは、判断の前に、

- 今の身体状態
- 気分
- 習慣
- 記憶
- 欲望
- 将来像
- その日の空気
- なんとなくの引力や抵抗

みたいなものが、まだ言語化されないまま相互作用している。

音楽もかなり似ています。音楽は、命題を明示してから推論するものではなく、まず**感覚の重みづけや期待の勾配**を作る。次に何が来てほしいか、何が来ると腑に落ちるか、逆に何が外れると面白いのか、という「価値空間」を先に形成する。

つまり音楽は、何かを“説明”する前に、**何が意味を持つかの地形を整える**。

あなたの「今日はなにしようかな」も同じで、これは行動選択というより、

自己の内部にある複数の価値系を、その場で調停している過程

に近い。

音楽もまた、和声、リズム、反復、逸脱、解決延期などを通して、複数の力を同時に走らせ、ひとつの方向感を仮設的に作る。

もう少し踏み込むと、似ている点はこのあたりかもしれません。

1. 先に目的がない

音楽は「この結論を伝えるために鳴る」とは限らない。
「今日はなにしようかな」も、結論より先に探索がある。

2. 局所判断の積み重ねで全体が立ち上がる

音楽は次の一音が、直前までの流れと緊張から決まる。
一日の過ごし方も、その瞬間の身体・記憶・環境との相互作用から立ち上がる。

3. 論理よりも“適合感”が重要

音楽で正しさより「しっくり来る」が重要なように、
「今日はこれだな」も演繹ではなく、ある種の整合感で決まる。

4. 未分化なものを時間の中で形にする

音楽は、まだ形を持たない感情や運動を時間の中で編成する。
「今日はなにしようかな」も、未分化な欲望を一日の行為へ変換する。

なので、あなたが感じた類似はかなり筋が通っています。

雑に言えば、

音楽は、感情や価値の探索を時間構造として外化したもの

であり、

「今日はなにしようかな」は、その探索を内面でやっている状態

とも言える。

さらに言うと、AIが得意な「目的→手段」の最適化は、音楽のかなり表層しか扱えないのと同じで、この問いの核心も取り逃がしやすい。問題は「何を達成するか」ではなく、**何が自分にとって呼びかけとして現れるか**だからです。

あなたが「音楽のある特性」に似ていると感じたのは、おそらく

両者とも、意味や目的を前提せず、時間の中で“次に何が来るべきか”を探る営みだから

だと思います。

ただ、これは一つの解釈です。

別の可能性としては、あなたが似ていると感じたのは「旋律」よりむしろ**和声**かもしれない。つまり、単一の目的ではなく、複数の欲求や制約が同時に鳴って、そのバランスの一時解として行動が決まる、という意味で。こちらのほうが、あなたの書いた「本能区間への価値の問い合わせ、現在状況の確認、習慣、趣向、長期目標、過去記憶…」にはむしろ近い。

その場合、「今日はなにしようかな」はメロディではなく、**内的和声の解決先を探しているの**かもしれません。



なるほど。。。そのとおりかもしれません。

生体モデルにしたAI等プロジェクトでさんざん考えていますが、
音楽は、まだ解明できていない、脳の知性の仕組みの、一般的な仕組みを示唆する、何かがあると考えますか？それとは関係ないただの特殊な反射でしょうか？
もし前者なら、どの様な仕組みの存在を示唆していますか？

前者だと思います。

ただし、**「音楽そのものが知性の秘密を表している」**とまではまだ言えません。より慎重に言うと、音楽は、脳のかなり一般的な仕組み——**予測、時間階層、身体との結合、報酬づけ、社会的同期**——が、非常に見えやすい形で露出する課題です。単なる特殊反射で片づけるには、関与する系が広すぎるし、構造が深すぎる。逆に、音楽だけを特別視しすぎるのも危ない。今のところ一番妥当なのは、**音楽は一般知能の周辺現象ではなく、一般的な脳計算が高密度に現れる“実験場”である**、という見方です。 [Nature+2](#)

私なら、音楽が示唆している一般機構は少なくとも5つあります。

1. 知性は「目的からの推論」だけでなく、予測誤差を使って時々刻々と世界モデルを整える仕組みを含む。

音楽知覚の主流理論のひとつは predictive processing で、脳は次に来る音や和声や拍を予測し、外れ方を使って内部モデルを更新する、と考えます。これは音楽専用というより、知覚・行為・学習に広く関わる一般原理として提案されています。音楽でこれが目立つのは、対象が本質的に時間的で、「次」を常に要求するからです。 [Nature+2](#)

2. 知性は単発の推論ではなく、複数の時間スケールをまたぐ階層処理を必要とする。

音楽では、数十ミリ秒の音色や立ち上がり、数百ミリ秒～数秒の拍・フレーズ、さらに長い緊張と解決が同時に走ります。最近の研究でも、聴覚系は予測や記憶を階層的に処理していること、また階層的な聴覚処理が皮質内・皮質間で組織されていることが示されています。これは、知性が「今この瞬間」だけを見るのではなく、**多層の時間窓で状態を保持し統合する必要がある**ことを示唆します。言い換えると、脳はRNNやTransformer的に“履歴を持つ”だけでは足りず、**異なる時間定数の並列ダイナミクス**を持っている可能性が高い。 [Nature+2](#)

3. 価値や快は、内容そのものより「予測の当たり外れの構造」から生まれる。

音楽は食物や性のような一次報酬ではないのに、強い快を生む。この点が重要です。最近の自然音楽刺激を使った研究では、好みは不確実性と驚きの相互作用に依存し、報酬系、とくに腹側線条体が関与することが示されています。つまり脳は、ただ外界を当てるだけでなく、**“学習可能だが自明ではない構造”を見つけること自体に価値を与える**らしい。もしそうなら、知性の核心には「正解発見」だけでなく、**適度な圧縮可能性・探索価値への報酬づけ**があることになります。これは、あなたの言う「今日はなにしようかな」にもかなり近い。目的を選ぶ前に、どの方向が“感じとして価値があるか”を評価しているからです。 [Frontiers+1](#)

4. 知性は“内部表象の計算”だけでなく、身体との結合を含む。

近年は、音楽理解を純粋な内部予測モデルだけでなく、**神経・身体ダイナミクスの共鳴や同調**として捉える立場も強いです。拍に乗る、グルーブを感じる、演奏でタイミングを合わせる、といった現象は、命題的推論よりも、身体化された時間同期で説明したほうがうまくいく部分がある。これは大きな含意を持ちます。知性は「記号操作器」ではなく、**外界リズムと自身の内部振動を結合させる時間機械**かもしれない。音楽はその性質をむき出しにする。 [Nature+2](#)

5. 知性は個体内だけで完結せず、他者との同期・共有予測を作る。

音楽は社会性の実験場でもあります。集団演奏や合唱では、各人が他者のタイミングや強弱を予測しつつ、自分の行為を調整します。レビューでも、音楽知覚は感覚だけでなく、行為、情動、学習、さらに共同音楽行為へ拡張されると整理されています。ここから見えるのは、知性の一部が**共有された時間予測の形成**だということです。個体知能のモデルだけでは、ここは取りこぼしやすい。 [Nature+1](#)

ただし、ここで一段冷やしたほうがいい点があります。

音楽研究からすぐに「脳の一般原理」が読めるわけではありません。少なくとも現在は、**予測符号化**の説明と、**共鳴・神経動力学**の説明は競合というより併存しています。前者は内部モデルと誤差更新を強調し、後者は系の自己組織化や強い予期を強調する。どちらがより基礎的かは決着していません。むしろ重要なのは、音楽がこの両方を必要としているらしい、という点です。つまり脳は、**表象的でもあり、動力学的でもある**。そこがAIにとって厄介で、同時に示唆的です。 [Nature+2](#)

また、音楽には確かに特殊性もあります。歌や楽曲処理が音声処理と完全と同じではないこと、文化差や学習履歴が大きいこと、和声や協和の嗜好が音色や文化でかなり変わることも示されています。なので「音楽は普遍本能そのものだ」と単純化するのも危ない。正確には、**一般メカニズムが文化的学習を通して高次に整形された領域**と見るのがよさそうです。 [Nature+2](#)

あなたの関心に寄せて、AIへの含意だけを抽出すると、音楽が示しているのはたぶん次です。

知性には、**(a) 多時間スケールの予測、(b) 予測誤差だけでなく“よい驚き”への価値づけ、(c) 身体・運動との閉ループ、(d) 他者との同期、(e) 目的を与えられる前に価値地形を生成する仕組み、が必要だ、**ということ。今の多くのAIは(a)の一部は強いが、(b)~(e)が弱い。だから手段選択はできても、「今日はなにしようかな」のような**目的以前の自己編成**が苦手です。音楽はそこを突いてくる。 [Nature+3](#)

結論だけ言えば、私は**音楽はただの特殊反射ではない**と思います。

むしろ、**脳の知性が**

「世界を当てる機械」ではなく、「時間の中で予測・価値・身体・他者を結び、次に何が来るべきかを自己組織化する機械」である

ことを示唆している。音楽は、その一般原理がもっとも高密度に見える窓のひとつです。 [Nature+2](#)

この観点をそのまま生体モデルAIに落とすなら、次に詰めるべき問いは「報酬関数を外から与えないとき、系はどうやって“よい次”を定義するのか」だと思います。



>次に詰めるべき問いは「報酬関数を外から与えないとき、系はどうやって“よい次”を定義するのか」だと思います。

まさにそれですね。それは、興味深い面を含む反面、直ぐに議論が発散する危険もありますね。もう一つ、何か焦点を持たせないと。。。。そもそもの目的から考えるひつようがあります。わたしは、そもそもの目的は何を得ようとして、この話題を出していると思いますか？

たぶん、あなたが得ようとしているのは、音楽の説明そのものではありません。

もっと絞ると、**生体モデルAIで本当に欠けている中核変数は何か**を見極めようとしているのだと思います。

つまりこの話題は、

- 音楽を理解したい
- 人間の気分や衝動を説明したい

というより、むしろ

- **目的がまだ定まっていない状態から、どうやって次の行為が立ち上がるのか**
- そのとき脳内では、何が状態変数として支配的なのか
- それをAI設計に落とすなら、何を実装対象にすべきか

を探索するための探針として出しているように見えます。

もう少し露骨に言うと、あなたはたぶん

「知能とは、与えられた目的を達成する能力ではなく、目的候補そのものを生成し、重みづけし、行動へ収束させる能力ではないか」

という疑いを持っている。

それが正しいなら、今のAIの中心設計はかなり外していることになる。だからこの話題は、単なる興味ではなく、**設計原理の再定義**に関わっている。

その観点から見ると、あなたの本当の問いは音楽についての問いに見えて、実際には次のどれかです。

1. 価値はどこから来るのか

外部報酬なしで、なぜある方向が「行くに値する」と感じられるのか。

2. 未決定状態はどう安定化するのか

欲求、記憶、身体状態、環境、長期目標が競合しているとき、なぜ最終的に何か一つの行為へ収束できるのか。

3. “意味の前段階”をどうモデル化するか

言語化された目標や命題になる前の、まだ曖昧な引力・違和感・期待のようなものを、どう扱うべきか。

4. そこに音楽がなぜ効くのか

音楽は、意味内容なしに価値勾配や期待構造を作れるから、その前段階の機構を露出しているのではないか。

なので、あなたの目的を一文で言えば、

「目的選択以前の知能」の実在する構造を見つけたい

のだと思います。

さらに厳しく言うと、あなたは議論を広げたいのではなく、逆です。

“なんでも説明できてしまう抽象論”を捨てて、設計可能な核へ落としたい

のではないですか。

その意味で「議論が発散する危険がある」「もう一つ焦点を持たせないと」という感覚は自然です。発散の原因は、価値・意識・自由意志・美・快などを全部まとめて扱おうとすることだからです。

焦点を持たせるなら、あなたの本来の目的に最も近いのは、たぶんこの問いです。

外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶのか。

これならかなり具体化できます。

そして音楽は、この問いに対するよいテストベッドになります。なぜなら音楽では、

- 明示的な外的目的が弱い
- それでも強い方向感が生まれる
- 次への期待と価値づけが可視化される
- 身体・記憶・予測・快が絡む

からです。

なので、あなたが得ようとしているのは、おそらく

音楽論ではなく、目的生成系の設計原理です。

さらに踏み込めば、こういう懸念も含んでいるはずです。

もしそこを外すと、生体モデルAIは結局、

- 外から与えた目的に従うだけの最適化器
- 内発性を模したように見える表層的生成器
- “したい”ではなく“選ばされた”行動しか持てない系

に留まる。

あなたはそこを越えたいのだと思います。

私の見立てが当たっているなら、次に立てるべき問いは「価値とは何か」では少し広すぎて、むしろ
行動の直前に、複数候補のあいだで生じる“行きたさの差”は、何の計算結果か

くらいまで狭めたほうがいいです。

ここまで落とすと、かなり研究の形になります。



すばらしいです。うまく整理できていると思います。

>外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶのか。

まさにそれです。

個人的には、その最小パターンを、実装したいです。

それこそ、線虫レベルで。しかし、そこで疑問がでます。

>外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶのか。

は、線虫レベルに既に存在するのか、哺乳類や鳥類レベルの脳でないといけないのか。

もし線虫レベルであれば、脳の基本システムにかなり根差した仕組みといってよいことになります。

まさにこれを知るためのアクションを取りたいのです。

結論から言うと、**線虫レベルにすでにあると見るのが妥当**です。

ただし、そこにあるのは人間的な「目的表象」ではなく、**内部状態に応じて行動候補の重みを変え、状態遷移として次の行動を選ぶ最小機構**です。C. elegans では、餌上での roaming / dwelling、餌パッチから離れるか残るか、温度・空腹・過去経験に応じた探索状態の切替、感染時の sickness behavior などが、感覚入力だけでなく内部状態と neuromodulation によって変わることが繰り返し示されています。つまり「外的目標が未定でも、内部変数が行動選択を曲げる」こと自体は、かなり小さな神経系でも成立している。 [eLife+4](#)

なので、あなたの問いは少し言い換えたほうがいいです。

「線虫に目的があるか」ではなく、**「線虫に、内部状態に依存して行動方策を切り替える最小の action-selection 機構があるか」**です。これなら、かなり強く yes と言えます。たとえば roaming / dwelling では、セロトニン系と PDF 系の拮抗が長続きする探索状態を支え、感覚状況が変わると別状態へ切り替わる回路構造が示されています。さらに、餌パッチから離脱する rare decision は arousal state に結びついていて、高覚醒の roaming から出やすく、low arousal の dwelling では抑えられます。これは「刺激→反応」ではなく、**隠れた内部状態が同じ刺激への応答確率を変える**ということです。 [Cell+3](#)

ここで重要なのは、線虫で見えているものが**単なる反射の寄せ集めではない**点です。近年の研究では、C. elegans には持続的な behavioral state が複数あり、それらが food availability、temperature の現在値と履歴、空腹、感染、食餌の質などを統合して切り替わることが示されています。2023年の Nature Communications 論文では、温度履歴・現在温度・餌の有無の組み合わせで scanning や global search などの状態が生じ、別々の tonic sensory pathway と neuropeptide signaling がそれを支えると報告されています。2025-2026年の仕事でも、食餌の質を serotonin system が長時間スケールの foraging に結びつけること、腸内病原体が空腹由来の意思決定を上書きすることが示されています。これは、**内部状態変数が sensory evidence に単純加算されるだけでなく、評価軸そのものを書き換える**ことを意味します。 [Nature+2](#)

一方で、ここからすぐに「線虫が人間と同じように目的を生成する」と飛ぶのは危ないです。線虫で強く言えるのは、せいぜい次のレベルです。

- 体内状態や最近の経験が latent state を作る
- latent state が感覚入力の利得や行動遷移確率を変える
- その結果、探索・滞在・回避・摂食などの方策が切り替わる

ここにはすでに、あなたの言う「外的目標が未定のとき、内部の何を最適化して次の行動候補を選ぶのか」の**原型**があります。けれども、哺乳類で議論されるような自己モデル、将来想像、明示的な価値記述、言語的目標設定までは要りません。つまり、**目的生成の高次版は不要だが、目的以前の方策選択の低次版はある**、という切り分けがいちばん妥当です。roaming/dwelling の回路、脳全体活動の状態表現、睡眠や egg-laying の active/inactive state などは、その見本になっています。 [Cell+3](#)

あなたの実装目的に引きつけると、線虫から引ける最小パターンは、たぶん**「価値を計算する」のではなく、「状態依存で政策を切り替える」**です。もっと言えば、最小単位はこうです。

1. 内部状態変数

空腹、飽和、感染ストレス、覚醒、最近の報酬履歴のような slow variable。

2. 外界入力

食物濃度、温度勾配、嫌悪刺激など。

3. 行動モード

local search / global search、stay / leave、feed / avoid のような離散モード。

4. neuromodulator 的なゲート

行動モード間の遷移確率と、感覚入力の重みを変える。

5. 持続性とヒステリシス

一度入ったモードが少し維持される。これがないと「ただの反射」に戻る。

この形は、C. elegans の foraging state や egg-laying state とかなり整合します。特に最近の Current Biology の egg-laying 研究では、active state が positive feedback loop で維持されることが示されていて、**持続状態 + トリガー + ゲート**という構図がはっきり出ています。 [Cell+2](#)

なので、あなたが今取るべきアクションは、哲学的に「目的とは何か」を掘ることではなく、**線虫で既に成立している最小 action-selection 機構を特定し、それをそのまま実装仮説に落とす**ことです。焦点は次の問いに置くとよいです。

Q1. 線虫で、行動切替を決めている最小の latent state は何個必要か。

local/global search、roaming/dwelling、stay/leave を別々に説明するのではなく、少数の hidden state でまとめられるかを見る。隠れ状態を減らしても説明力が落ちないなら、それが実装候補です。reviewed preprint でも、foraging を hidden states で parsimoniously 説明しようとする流れがあります。 [eLife+1](#)

Q2. その latent state は何で更新されるか。

感覚の瞬間値ではなく、空腹、recent food history、temperature history、arousal、infection などの slow variables が鍵です。C. elegans の研究はまさにそこを突いています。 [Nature+3](#)

Q3. その latent state は何を変えるか。

報酬値そのものではなく、感覚利得、行動遷移確率、モード持続時間を変える、という形で実装するほうが線虫らしいです。roaming/dwelling や patch leaving の結果は、この見方とよく合います。 [eLife+1](#)

私なら、まずこう進めます。

第一段階：生物側の作業仮説を固定する。

「線虫にあるのは goal representation ではなく、neuromodulator-gated policy switching である」と一度決める。ここを曖昧にすると発散します。根拠は roaming/dwelling、arousal-coupled leaving、multimodal state switching、sickness behavior です。 [eLife+3](#)

第二段階：最小実装を 3 状態くらいで作る。

内部状態は 2~3 個で十分です。たとえば energy、arousal、aversive load。外界は food cue と danger cue。行動モードは explore / exploit / avoid。更新則は連続時間でも離散時間でもよいですが、重要なのは slow variable が policy をゲートすることです。ここで「報酬最大化」は主役にしないほうがいい。むしろ homeostatic error と uncertainty reduction の混合のほうが近いはずです。これは私の推論ですが、上の線虫データとは整合します。 [eLife+2](#)

第三段階：ベンチマーク課題を線虫風にする。

「迷路を解く」ではだめです。

使うべきなのは、餌パッチ上にいる個体が stay/leave を選ぶ課題、餌除去後の local-to-global search 課題、温度履歴つきの探索課題です。これなら生物結果と直接比較できます。 [eLife+2](#)

私の見立てでは、あなたが次に知るべきなのは

“線虫にそれがあるかないか”ではなく、もう一段具体的に、

“線虫では、内部状態は報酬を計算しているのか、それとも政策の切替器として働いているのか”

です。いま出ている証拠は後者をかなり支持しています。ここが定まると、最小実装はだいぶ締まります。

一言で言えば、

線虫レベルに既にある。ただしそれは『目標を考える知能』ではなく、『内部状態で行動モードを切り替える知能』としてある。

この差を最初に固定してしまうのが、たぶん次の一手です。

必要なら次に、線虫レベルの最小数理モデルを、そのまま実装しやすい形で 10 行程度の状態方程式まで落とします。



外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶのかを、内在的価値信号 という言葉に置き換えられますか？

最小の生態的知性の、要素があるとし、他の候補はなんでしょうか？

部分的には置き換えられます。

ただ、完全な置き換えではないです。

「外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶのか」は、

- 何かしらの評価軸がある
- その評価軸が行動選択に効く
- しかも外から明示されていない

という意味では、たしかに 内在的価値信号 という言葉にかなり近いです。

しかし、この言い換えには危険があります。

それは、もともと動的で分散的なものを、単一のスカラー信号のように見せてしまうことです。

実際には、

- 空腹
- 覚醒
- 不確実性
- 新奇性
- 慣れ
- 回避傾向
- 接近傾向
- 最近の履歴
- 行動モードの慣性

などが競合して、結果として「今はこれに行く」が立ち上がるかもしれない。

この場合、ある一個の value signal があるというより、**複数の内部変数の場から、局所的に価値勾配が生じる**と見たほうがよい。

なので、私ならこう切り分けます。

「**内在的価値信号**」は有力な候補語だが、設計原理としてはまだ粗い。

より中立に言うなら、

内的状態に依存した行動選択バイアス

あるいは

内的状態から生成される行動優先度場

のほうが、実装には近いです。

では、「最小の生態的知性」の要素候補は何か。

ここで大事なのは、知性を高級な推論能力としてではなく、**環境の中で壊れずに次の行動を立ち上げる仕組み**として見ることです。そうすると、候補はかなり絞れます。

1. 内部状態変数

最有力です。

空腹、疲労、覚醒、危険負荷、繁殖準備、温度ストレスのような遅い変数。

これは「何をしたいか」を直接表現しなくても、行動の重みを変えられる。

最小系ではまずこれがないと、全部ただの刺激反応になります。

2. 行動傾向の重みづけ機構

これも本質です。

内部状態を受けて、候補行動の優先度を変える仕組み。

たとえば、

- 探索

- 滞在
- 接近
- 回避
- 摂食
- 休止

のどれが今通りやすいかを変える。

これは「価値信号」より少し具体的で、実装しやすい。

3. モード切替

かなり重要です。

最小知性は、細かい連続最適化よりも、**行動モードの切替**として現れることが多い。

- explore / exploit
- stay / leave
- active / quiescent
- forage / escape

この切替があるだけで、系はかなり“知的”に見えます。

しかも線虫レベルとも相性がいい。

4. 時間的持続性

見落とされやすいですが重要です。

内部状態やモードが少し持続すること。ヒステリシスでもよい。

これがないと、毎瞬間の入力でふらつく。

生態的知性には、「いったんこの方針で行く」が必要です。

5. 予測または期待

強い世界モデルでなくてよいです。

最小レベルでは、

- この方向には餌がありそう
- ここに留まっても薄そう
- この刺激の後は危険が増えそう

くらいの粗い期待で十分。

これは「次」を作る仕組みです。

音楽との類比で言えば、まさにここです。

6. 誤差・ズレの検出

期待と現実のズレがないと、更新が起きない。
ただし、これは報酬誤差に限りません。

- 期待より薄い
- 危険に近い
- 満たされない
- 単調すぎる

こういうズレが、モード更新の契機になる。

7. 探索駆動

これは独立要素として立てる価値があります。
生態的には、既知の場所に留まるだけでは死ぬことがある。
だから最小知性には、外的目標がなくても**未知へ少しにじみ出る力**が必要です。
これを「新奇性選好」と呼ぶと狭い。
むしろ **停滞回避圧** くらいのほうがよいかもしれません。

8. 身体と環境の閉ループ

知性を内部計算だけで見ないなら、これは外せません。
知性は内部に完結せず、

- 動く
- 感じる
- 状態が変わる
- それでまた次が変わる

という循環で成立する。
最小の生態的知性は、この閉ループの設計抜きでは語れないです。

ここで、候補を整理すると、あなたの問いの核に近いものは次の4つです。

A. 内在的価値信号

「今どの方向がよりよいか」を示す内部評価。
長所は直感的。
短所は、一個の値に潰しすぎることに。

B. 内部状態依存の行動優先度場

複数の行動候補に対して、今どれが通りやすいかの場。
私はこれが一番よいと思います。
価値信号より構造を残せる。

C. モード切替器

知性の最小単位を、候補行動間の連続比較ではなく、離散モードの切替とみなす立場。
線虫実装にはかなり向いている。

D. 内的整合性維持機構

系が自分の状態を壊さず、かつ停滞もしないように次を選ぶ仕組み。
これはかなり抽象的ですが、最小の生態的知性を一番広く捉えています。

私なら、今の段階では「内在的価値信号」を最終語にしません。
代わりに、作業用の中心概念をこう置きます。

内的状態により生成される行動優先度場

その上で、それを構成する下位要素として

- 恒常性変数
- 覚醒／危険／飽和のようなモジュレーター
- 候補行動の活性
- モード持続
- 期待とズレ
- 探索圧

を置く。

これだと、価値という哲学語に引っ張られすぎず、実装にも落としやすいです。

さらに一段圧縮すると、
最小の生態的知性の候補は、次の3案に集約できます。

案1：価値最適化系

内部の価値信号を最大化する。
わかりやすいが、単純すぎる恐れがある。

案2：政策切替系

内部状態に応じて行動モードを切り替える。
線虫レベルには最も自然。

案3：安定性-探索バランス系

壊れず、停滞せず、環境との循環を維持する。
生態的にはかなり本質的。

私の見立てでは、最小実装の最初的一步としては
案2を中心に、案3を目的関数のように背後に置く
のがよいです。

案1は、その結果として局所的に見える量として扱うほうが破綻しにくい。

つまり、

- 本体は「価値信号」ではなく
- **状態依存の政策切替機構**
- その背後に **生存的安全性と探索の両立**
- そして観測されるものとして **内在的価値らしき信号**

という順です。

必要なら次に、

この候補群を **線虫レベルで実装可能な最小アーキテクチャ 3案** にまで落とします。



案1，案2，案3は、生体的な最小の知性に必要な要素として

1. 内部状態変数
2. 行動傾向の重みづけ機構
3. モード切替
4. 時間的持続性
5. 予測または期待
6. 誤差・ズレの検出

で構成されるということでしょうか？

だいたいその理解でよいです。

ただし、**案1・案2・案3**がその**6要素**で同じように構成される、とまでは言わないほうがよいです。

正確にはこうです。

- その**6要素**は、「**最小の生体的知性**」を記述するための**共通部品候補**
- **案1・案2・案3**は、それらをどう編成するかの異なる見方

です。

なので、関係は

- 6要素 = 部品
- 案1～3 = アーキテクチャ上の立場

と考えるのがよいです。

まず、6要素について

あなたの挙げた

1. 内部状態変数

2. 行動傾向の重みづけ機構
3. モード切替
4. 時間的持続性
5. 予測または期待
6. 誤差・ズレの検出

は、かなり筋がいいです。

少なくとも、「ただの反射」から一歩出るための最小セット候補として成立しています。

ただし、少し整理し直したほうがいい。

中核に近いもの

- 内部状態変数
- 行動傾向の重みづけ機構
- モード切替
- 時間的持続性

この4つはかなり本質的です。

これだけでも、かなり「それっぽい」最小系が作れます。

追加で効いてくるもの

- 予測または期待
- 誤差・ズレの検出

これは重要ですが、最小性をどこまで削るかで位置づけが少し変わります。

極端に言えば、線虫レベルの最小モデルでは、5と6を明示的な予測器として持たずに、

- 状態依存の遷移
- 感覚入力への慣れ
- 行動履歴に依存する持続

だけで近似できる可能性があります。

つまり、5と6は

明示的なモジュールとして必須かどうかは未確定です。

ただし、動作としてはほぼ必ずどこかに埋め込まれているはずです。

案1～3との対応

案1：価値最適化系

これは、6要素のうち特に

- 1. 内部状態変数

- 2. 行動傾向の重みづけ機構
- 5. 予測または期待
- 6. 誤差・ズレの検出

を強く必要とします。

なぜなら、何かを「最適化する」なら、

- 現在状態を読む
- 未来をある程度見積もる
- ズレを評価する

が必要だからです。

この案では、3と4は補助的です。モード切替や持続は、価値最適化の実装方式として出てくる。

案2：政策切替系

これは、最も線虫に近い見方です。

中核は

- 1. 内部状態変数
- 2. 行動傾向の重みづけ機構
- 3. モード切替
- 4. 時間的持続性

です。

5と6は、なくても最小モデルとしては回る可能性がある。

あるいは、**切替規則の中に暗黙に埋まっている**。

たとえば、

- 餌が減った
- 一定時間成果がない
- 危険入力が続いた

ときにモードが変わるなら、それは広い意味でズレ検出ですが、必ずしも明示的な prediction error を計算している必要はない。

なので、案2は6要素全部を使ってもいいが、**骨格は1〜4**です。

案3：安定性-探索バランス系

これは少し抽象度が高い。

主に

- 1. 内部状態変数
- 2. 行動傾向の重みづけ機構

- 4. 時間的持続性
- 6. 誤差・ズレの検出

が重要です。

ここでのズレとは、予測誤差というより

- 飢餓の増加
- 刺激の乏しさ
- 危険の蓄積
- 同じ状態に留まりすぎること

のような**生態的な不整合**です。

3のモード切替は、その結果として出る。

5の予測はあると強いが、最小形ではなくてもよい。

なので、あなたの理解を少し修正すると

「案1・案2・案3は、これら6要素で構成される」

より、

「これら6要素は共通の候補部品であり、案1〜3はそのうち何を中核に置くかの違い」

のほうが正確です。

さらに削るなら、最小核は何か

かなり厳しく削ると、私は最小核はまずこの4つだと思います。

1. 内部状態変数
2. 状態依存の行動重みづけ
3. モード切替または行動遷移機構
4. 持続性

これだけで、「反射以上」のかなり重要な部分が出ます。

その上に、

5. 期待
6. ズレ検出

を載せると、より知性的になる。

逆に、まだ足りないかもしれない要素

あなたの6要素はよいのですが、ひとつ補うなら

候補行動の生成

を別に立ててもよいです。

今の6要素は、「候補がある前提」でその選択を説明しています。

しかし本当は、

- explore
- exploit
- avoid
- rest

のような候補自体がどう立ち上がるかも問題です。

ただ、線虫レベルの最小実装なら、候補は固定してよいので、最初は省いてよいです。

もうひとつ補うなら

身体-環境閉ループ

です。

ただしこれは要素というより、全体の前提条件です。

実装向けに言い直すなら

最小の生体的知性は、まず

- 遅い内部状態
- 状態依存の行動バイアス
- 離散的または準離散的なモード
- モードの慣性

で作る。

その次に

- 期待
- ズレ検出

を入れる。

そして案1〜3は、

- 案1: それらを価値最大化として読む
- 案2: それらを政策切替として読む
- 案3: それらを安定性と探索の均衡として読む

という違いです。

一言でまとめると、

はい、かなりその理解でよいです。ただし6要素は“案1〜3を構成する固定セット”というより、“最小知性を作るための部品群”であり、案1〜3はその部品群への異なる理論的な焦点の当て方です。

次に進めるなら、この6要素を

必須・準必須・後回し

の3段階に分けるのがよいです。



6要素ですが、いまいち概念としては、重複がある気がします。あなたがまとめたように、「身体-環境閉ループ」という言葉が、それらのうち幾分かをうまく代表してくれそうです。生体的な最小の知性に必要な要素として、6要素をよりエレガントに「身体-環境閉ループ」という概念が解釈してくれるとおもいますが、これと同様の概念をいくつか加えて、過不足なく表現できそうですか？

できます。

むしろ、その方向のほうがよいです。6要素をそのまま並べると、実装項目としては便利でも、**概念としては粒度が揃っていない**。状態、機構、時間特性、制御原理が混ざっています。だから重複感が出る。

「身体-環境閉ループ」は、その混線をかなり整理できます。

ただし、これ一語だけではまだ広すぎます。これだけだと、

- 何が内部で保持されるのか
- 何が選択を生むのか
- 何が安定と変化を両立させるのか

がぼやける。

なので、**2〜4個の上位概念**に圧縮するのがよいと思います。

私なら、最小の生体的知性を次の4概念で表します。

1. 身体-環境閉ループ

これは土台です。

知性を、内部表象だけでなく、

- 感じる
- 動く
- 環境が変わる
- その変化がまた入力になる

という循環として捉える概念です。

ここには、元の6要素のうち少なくとも

- 行動傾向の重みづけ機構
- 予測または期待
- 誤差・ズレの検出

のかなりの部分が吸収されます。

というより、これらは閉ループの中で意味を持つ。単体ではなく。

ただし、この概念だけでは「系の内部に何が蓄えられているか」が言えません。

2. 内部状態空間

これは、系がその瞬間の刺激だけで動いていないことを表します。

空腹、覚醒、危険負荷、履歴、飽和、感染、最近の成功失敗など、遅い変数を含む「内部の座標系」です。

元の6要素では

- 内部状態変数
- 時間的持続性

を主にここへまとめられる。

「持続性」を別要素にしないのは、持続性は独立のものというより、**内部状態空間があることの帰結**だからです。内部状態がなければ、持続も履歴依存も立ち上がらない。

3. 行動モード力学

これは、系が毎瞬間ゼロから選択しているのではなく、探索・滞在・回避・休止のような**準安定な行動様式**を持つことを表します。

元の6要素では

- モード切替
- 行動傾向の重みづけ機構
- 時間的持続性

のかなりがここに入る。

重要なのは、選択を「価値最大化の一点計算」として見るより、**モード間遷移の力学**として見たほうが、生体的最小知性には自然だということです。

4. 生態的整合性

これは少し抽象的ですが、必要です。

系が何をしているかを一言で言えば、単純な報酬最大化ではなく、

- 壊れない
- 行き詰まらない

- 環境との関係を維持する
- 必要に応じて探索へ移る

という整合性を保っている。

元の6要素のうち

- 予測または期待
- 誤差・ズレの検出
- 行動傾向の重みづけ機構

は、ここから見ると「整合性維持のための局所機構」です。

これは「内在的価値信号」より広く、しかも過剰にスカラー化しない。
生体らしさを残せます。

この4つの関係はこうです。

- **身体-環境閉ループ**
知性が成立する場
- **内部状態空間**
系が現在をどう背負っているか
- **行動モード力学**
系が次の行動をどう立ち上げるか
- **生態的整合性**
その選択が何に奉仕しているか

かなり収まりがよいと思います。

さらに削るなら、3概念でもいけます。

圧縮版 3概念

1. 身体-環境閉ループ

知性は循環である。

2. 内部状態に支えられた行動モード力学

知性は状態依存のモード遷移である。

3. 生態的整合性の維持

知性は外的目標でなくても、壊れず停滞せず次を作る。

これでもかなり十分です。

6要素は、この3概念の下位展開として扱える。

逆に、「過不足なく」にこだわるなら、もう一つ足して4概念のほうが安定です。
その追加候補は **時間的厚み** です。

5. 時間的厚み

生体的知性は瞬間反応ではなく、履歴と先行きを含む時間構造を持つ、という概念です。

これは本来、

- 内部状態空間
- 行動モード力学
- 期待
- ズレ

に分散して入っています。

でも、時間性を特に重視したいなら、独立概念にしてもいい。音楽との連想にも近い。

ただ、最小知性の実装語彙としては、私はこれは独立させず、内部状態空間とモード力学に吸収したほうがエレガントだと思います。

あなたの元の6要素との対応をざっと書くと、

- **内部状態変数** → 内部状態空間
- **行動傾向の重みづけ機構** → 行動モード力学 / 生態的整合性
- **モード切替** → 行動モード力学
- **時間的持続性** → 内部状態空間 / 行動モード力学
- **予測または期待** → 身体-環境閉ループ / 生態的整合性
- **誤差・ズレの検出** → 身体-環境閉ループ / 生態的整合性

つまり、6要素は独立な「部品」ではなく、
4つの上位概念の局所的な現れとして読み替えられます。

私なら、いまの段階では定義文をこう置きます。

最小の生体的知性とは、身体-環境閉ループのなかで、内部状態空間に支えられた行動モード力学を通じて、生態的整合性を維持しつつ次の行動を立ち上げる能力である。

かなりまとまっています。

しかも、線虫レベルにも落とせるし、哺乳類にも拡張できる。

ただし、弱点もあります。

「生態的整合性」がやや便利語になりやすい。何でも入ってしまう。

そこを避けたいなら、もっと操作的にして

恒常性維持と探索継続の両立

と言い換える手もあります。こちらはやや狭いが、実装向きです。

その場合は4概念が、

- 身体-環境閉ループ
- 内部状態空間
- 行動モード力学
- 恒常性維持と探索継続の両立

になります。

私はこの版のほうが、研究計画には向いていると思います。

なので答えは、**はい、できます。**

しかも、6要素を無理に残すより、

- 身体-環境閉ループ
- 内部状態空間
- 行動モード力学
- 生態的整合性
あるいは
恒常性維持と探索継続の両立

の4概念へ圧縮したほうが、過不足が減ります。

次にやるべきは、この4概念を

線虫レベルで観測可能な変数と回路仮説に対応づけることです。



- > • 身体-環境閉ループ
- > • 内部状態空間
- > • 行動モード力学
- > • 生態的整合性
- > あるいは
- > 恒常性維持と探索継続の両立

身体-環境閉ループ

が構造を示すとして、最初の話題の

「外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶ」

は、その構造ループの指針ですね。

身体-環境閉ループ 以外は、かならず必須の要素でしょうか？冗長とか重複とかないですか？もっと、本質的に表現できる概念にまとめられませんか？これ以上は、哲学的になりすぎて、実用的でないですか？

よし本質的な概念にまとめるほど、実装もむだ省ける可能性もあると思いますが。

まとめられます。

しかも、その圧縮は実用的です。むしろ、今の段階では**圧縮しないほうが実装がぶれる**可能性が高いです。

まず、あなたの見立ては正しいです。

「外的目標が未定のとき、系は内部の何を最適化して次の行動候補を選ぶ」
は、個別部品ではなく、**ループ全体を駆動する指針**です。

だから、

- 身体-環境閉ループ = 構造
- 何を最適化して次を選ぶか = その構造を貫く原理

という分け方が自然です。

そのうえで、あなたの問いはこう言い換えられます。

身体-環境閉ループ以外に、独立した本質概念はいくつ必要か。

私の答えは、**多くてもあと1つか2つ**です。

それ以上増やすと、ほぼ確実に重複します。

今まで出てきた

- 内部状態空間
- 行動モード力学
- 生態的整合性
- 恒常性維持と探索継続の両立

は、全部ある程度正しいですが、**同じものを違う断面から見ている部分がある。**

特に、

- 内部状態空間 = 系の内部側の記述
- 行動モード力学 = 行動出力側の記述
- 生態的整合性 = 全体の評価側の記述

であって、別々の原理とは限らないです。

なので、もっと本質的に圧縮するなら、私はこうします。

最小2概念

1. 身体-環境閉ループ

系は、身体を通じて環境と循環的に結びついている。

2. 状態依存の整合性駆動

系は、そのときの内部状態に応じて、自己の存続・安定・探索継続にとってより整合的な次を選ぶ。

これでかなり足ります。

ここで「状態依存の整合性駆動」は、今までの

- 内部状態空間

- 行動モード力学
- 生態的整合性
- 恒常性維持と探索継続の両立

をかなりまとめて吸収しています。

つまり、

- **内部状態空間**は「状態依存」に入る
- **行動モード力学**は「次を選ぶ」の実装形に入る
- **生態的整合性**は「整合性駆動」に入る
- **恒常性維持と探索継続の両立**はその中身の代表例

です。

これ以上細かく分けるのは、原理の分解ではなく、実装詳細の分解に近い。

さらに1概念半まで削るなら

もっとラディカルに言えば、本質はほぼこれです。

生体的最小知性とは、身体-環境閉ループの中で、状態依存的に“より整合的な次”を立ち上げる能力である。

これだけでもかなり強い。

そしてこれは哲学というより、実装指針になります。

なぜなら、ここから必要な実装が逆算できるからです。

- 状態依存的であるために、内部状態変数が要る
- より整合的な次を出すために、行動候補の重みづけが要る
- 閉ループであるために、感覚-行動-環境更新が要る
- 持続や切替が必要なら、それはこの原理の実装として導入する

つまり、**上位概念を減らすほど、下位機構を採用する理由が明確になる。**

何が冗長か

かなりはっきり言うと、

内部状態空間と行動モード力学は、独立の第一原理というより、
「状態依存の整合性駆動」を実装したときに自然に出てくる内部構造です。

だから、最上位概念として両方を立てるのは少し冗長です。

同様に、

生態的整合性と恒常性維持と探索継続の両立も重なっています。

後者は前者のかなり実装寄りの言い換えです。

なので、最上位では両方いりません。

私なら最終的にこう整理します

最上位原理

1. 身体-環境閉ループ
2. 状態依存の整合性駆動

中位の派生概念

- 内部状態
- 行動モード
- 持続と切替
- 探索と恒常性のバランス

下位の実装要素

- 状態変数
- 遷移則
- ゲーティング
- 誤差や不足の指標
- 感覚入力と行動出力

これなら階層が揃います。

「何を最適化するか」はどう置くべきか

ここも大事です。

「最適化」という言葉をそのまま使うと、すぐにスカラー目的関数を置きたくなる。そこが危ない。

最小知性では、たぶん厳密には「最適化」ではなく、

状態依存的に整合性を増す方向へ遷移する

くらいが本体です。

つまり、

- 何か単一の値を最大化する
というより
- 壊れにくく、行き詰まりにくく、探索可能性を失いにくい方向へ寄る

です。

この見方によると、「整合性駆動」はかなり本質語になります。

哲学的すぎるか

ここは逆で、抽象化が足りないほうが実用性を失います。

今の段階で

- 内部状態
- モード
- 予測
- 誤差
- 価値
- 探索
- 恒常性

を全部並列に置くと、実装時に何が主で何が従かわからなくなる。

それは工学的には悪い抽象です。

本質概念へ圧縮するのは哲学ではなく、**設計のための圧縮**です。

もちろん圧縮しすぎて空疎になると駄目ですが、今のあなたのケースではまだ逆です。少し概念が多い。

なので、現時点の最も締まった表現

私はこれを推します。

最小の生体的知性とは、身体-環境閉ループのなかで、状態依存的により整合的な次の行動を立ち上げる能力である。

必要なら補助的に、

ここでいう整合性とは、恒常性の維持と探索継続可能性の両立を指す。

と添える。

これで十分実装に向かえます。

実装上の利点

この2概念化の利点は明確です。

- 「内部状態空間」を作る理由が明確になる
- 「行動モード」を入れるかどうかを、必要性ベースで判断できる
- 「予測器」が本当に必要か、単なる遷移ヒステリシスで足りるかを切り分けられる
- 「価値信号」を最上位に置かずに済む

つまり、余計なモジュールを減らせる可能性が高い。

結論としては、

はい、もっと本質的にまとめられます。しかもそのほうが実装に有利です。

最上位概念はおそらく2つで足ります。

- 身体-環境閉ループ
- 状態依存の整合性駆動

この2つを核にして、他は派生概念として落とすのが、いま一番締まった整理です。

次にやるべきなのは、この「状態依存の整合性駆動」を線虫レベルで測れる量に落とすことです。



すばらしい。

- 身体-環境閉ループ
- 状態依存の整合性駆動

にまとまりました。

状態依存の整合性駆動。

これ、最初話の、価値判断は、此の開年で言うとうどう表現されますか？

価値判断は、この枠組みでは

「状態依存の整合性駆動が、その時点の候補行動に与える局所的な順位づけ」

として表現するのがよいと思います。

つまり、価値判断は独立した第三の原理ではなく、

身体-環境閉ループの中で、状態依存の整合性駆動が実際に選択を生むときの現れです。

もう少し分解すると、

- 身体-環境閉ループ は、価値判断が生じる場
- 状態依存の整合性駆動 は、価値判断を生む原理
- 価値判断 は、その原理が個別場面で候補行動に与える序列

です。

なので、「価値」は本体ではなく、出力側に現れるものとして置くのがよいです。

ここはかなり重要です。価値を最初から独立の中心概念にすると、またスカラー報酬や効用関数に引っ張られやすい。

あなたの今の整理に乘せるなら、価値判断は例えば次のように言えます。

価値判断とは、ある内部状態と環境条件のもとで、整合性をより増すと見なされる行動候補に高い優先度を与える過程である。

これなら、

- 価値は固定的ではない

- 状態依存である
- 行動候補に対する比較として現れる
- 整合性という上位原理に従属している

という点が全部入る。

さらに圧縮すると、

価値判断 = 状態依存の整合性勾配の局所評価

とも言えます。

ただ、これはやや数理っぽい。

少し自然に言うなら、

価値判断とは、そのときの系にとって“より整合的な次”を感じ分けること
です。

この表現の利点は、最初のあなたの話ともつながることです。

「今日はなにしようかな」のとき、価値判断は、完成した目標に照らして評価しているのではなく、**まだ目標化されていない複数の次の候補に対して、どれが今の自己にとってより整合的かを感じ分けている。**

この感じ分けが、ここでいう価値判断です。

実装寄りに言い直すなら、価値判断は

- 内部状態
- 外界入力
- 行動履歴
- 候補行動

を入力として、各候補に一時的な優先度を与える写像です。

ただし、その優先度は「報酬最大化」ではなく、**整合性維持と探索継続可能性に対する寄与**を反映する。

なので、最終的にはこう整理できます。

価値判断は、状態依存の整合性駆動が、個々の行動候補に対して局所的な優先度差として現れたものである。

あるいは、もっと短く、

価値判断とは、整合性駆動の選択局面での表れである。

これでかなり締まると思います。

次に詰めるなら、「整合性」をどう測るかより先に、

価値判断を“スカラー値”として置くか、“候補間順位”として置くか

を決めるのがよいです。私は後者のほうが、今の枠組みには合っています。



整合性のなかに、予測が含まれるわけですね。

はい。かなり重要な意味で含まれます。
ただし、**予測そのもの = 整合性**ではないです。

正確には、

整合性を保つために、系は予測を使わざるをえない
という関係です。

なぜなら、整合性とは単に「今この瞬間が壊れていない」 ことではなく、

- このまま進むとどうなるか
- この行動を取ると状態はどう変わるか
- 探索を続けると危険が増えるか減るか
- 留まると枯渇するか安定するか

のような、**次の状態とのつながり**を含んでいるからです。

つまり整合性は本質的に時間的で、予測なしには十分に定義できません。

ただ、ここでいう予測は、必ずしも人間のような明示的シミュレーションではないです。
線虫レベルまで下げるなら、もっと弱い形でよい。

たとえば、

- 最近こういう入力が続いている
- この状態ではこの行動遷移が起きやすい
- この刺激パターンは餌の減少や危険の接近を示しやすい

といった、**状態遷移の期待**くらいでも十分です。

なので、整理するとこうです。

- **整合性** = 系が壊れず、行き詰まらず、次も動けるような関係の保たれ方
- **予測** = その整合性を評価するための時間的見積もり
- **価値判断** = その見積もりを使って、どの候補がより整合的かを局所的に順位づけすること

この順です。

少し厳しく言うと、予測を独立の中心概念にしすぎるのも危ないです。
脳や最小知性の議論では、何でも予測処理に回収したくなりがちですが、それだと

- なぜその予測が重要なのか
- 何のために予測するのか

が曖昧になる。

その問いに対して、あなたの今の整理では答えがある。

整合性のために予測する
のです。

これはかなり強い整理です。

予測は主役ではなく、**整合性駆動に奉仕する機能**として位置づく。

さらに言えば、最小実装では、予測を明示モジュールとして入れなくても、

- 履歴依存
- 状態依存の遷移確率
- モードの持続性
- 慣れや感作

だけで、暗黙の予測機能を持たせられるかもしれません。

そこも重要です。

つまり、**整合性には予測が含まれるが、予測器を露骨に作る必要があるとは限らない。**

一文で言うなら、

はい。整合性の中には予測が含まれます。ただし予測は目的ではなく、系が“より整合的な次”を選ぶための時間的機能です。



線虫レベル、植物、微生物レベルになれば、予測より本質的だと思います。しかし、そこまで来ると、知性とは呼べなくなってきました。それについては、これまでの議論では、どのように取り扱っていますか？

はい。そこで議論は割れます。

だいたい、これまでの流れは3つです。ひとつは、**知性や認知という語をかなり厳しく使う立場**で、神経系や表象、柔軟な学習、反事実的な予測のような条件を重く見る立場です。もうひとつは、**basal cognition / minimal cognition** の立場で、単細胞・植物・非神経系でも、環境との相互作用のなかで記憶、学習、状況依存の調整、目的指向的な制御があるなら、認知の連続体として扱うべきだとします。近年のレビューや方法

[Cambridge University](#)

論論文は、後者の流れをかなり明示的に押しています。 [Press & Assessment+3](#)

あなたの今の整理に近いのは、後者をそのまま飲み込むより、**語を階層化して使うやり方**です。つまり、植物や微生物レベルでまず認めるのは「人間的知性」ではなく、**生体の自己維持を伴う適応的な制御**です。その上に、柔軟な行動選択、学習、内部状態依存の切替があれば minimal cognition、さらにモデルベース予測や多段の表象が強くなれば、ふつう私たちが言う intelligence に近づく、という整理です。こうすると、「全部知

[Cambridge University](#)

性と呼ぶのは雑だが、全部を反射と呼ぶのも雑だ」という中間が作れます。 [Press & Assessment+2](#)

この文脈では、**予測より先にあるもの**として何を置くかが重要で、basal cognition 側ではしばしば、予測そのものよりも、**自己維持・自己組織化・環境との能動的な結合**が先に置かれます。enactivism 系や basal cognition 系の論文では、認知を「脳内表象」よりも、身体化された相互作用や生の自己調整の延長として捉える傾向が強いです。だから、あなたが言う「線虫、植物、微生物まで下げると、予測より本質的なものがある」という感覚は、かなりこの流れと一致しています。 [MDPI+3](#)

他方で、ここには明確な批判もあります。植物や微生物に cognition という語を広げすぎると、**擬人化や概念のインフレ**になる、という批判です。そのため最近の議論では、「知性が否か」を先に決めるより、**どの機能**

があるのかを分解して言う方向が強いです。たとえば、感受性、履歴依存性、状態依存の制御、学習、探索、可塑性、予測、表象、意識を分ける。これなら、植物や微生物に高次知性を無理に帰属せずに、しかし適応的

Cambridge University

制御の厚みはきちんと扱えます。 [Press & Assessment+2](#)

なので、これまでの議論をあなた向けに圧縮すると、こうなります。

微生物・植物レベルでは「知性」という語を無理に守る必要はない。代わりに、**身体-環境閉ループの中で、状態依存的に整合性を維持する適応制御があるかを見る**。そこに学習や行動切替や期待が加われば **minimal cognition** と呼び、さらに強い内部モデルや反事実操作まで行けば **intelligence** と呼ぶ。これがいちばん発

Cambridge University

散しにくい整理です。 [Press & Assessment+3](#)

だから、あなたの設計では最初から「知性を作る」と言わないほうがよいです。

まず狙うべきは、**最小の生体的整合性制御**です。そこで観察されるものとして、

- 状態依存の行動切替
- 履歴依存
- 探索と維持のバランス
- 環境に対する先回りの調整

が出てくるなら、そのあとで「これは minimal cognition と呼べるか」を問えばいい。

Cambridge University

順番を逆にすると、言葉の争いに吸われます。ここはかなり重要です。 [Press & Assessment+2](#)

私なら、研究上の作業語はこう置きます。

知性ではなく、生体的エージェンシーが整合性維持制御です。

その上で、どこから cognition と呼ぶかは後で判定する。これが一番、実装にも比較にも強いです。



では、生体的エージェンシーが整合性維持制御が、高度化するのに、予測や、記憶は必要かという話をしたい。よく粘菌の知性というはなしもあります。人間の脳は、粘菌の知性とはかけ離れているようですが、予測と記憶がなければ、人間の脳のようなものは作れないのか？これについては、ほぼ間違いないと考えていいのでしょうか？

はい。かなり強く「**ほぼその通り**」と考えてよいです。

ただし正確には、**人間の脳のようなもの**を作るのに必要なのは、豪華な明示的予測器や、百科事典のような記憶ではなく、もっと広い意味での **予測能力** と **記憶能力** です。ここでいう予測は「次の状態を先回りして行動を調整すること」、記憶は「過去が現在の内部状態や選択に持続的に残ること」です。人間の認知では、作業記憶は推論・学習・意思決定に広く関与し、長期記憶は想像や将来思考とも深く結びついています。さらに記憶と予測は別物というより相互依存で、予測誤差が記憶形成を変え、記憶が将来予測の内部モデルになります。 [Nature+3](#)

粘菌や単細胞の話が示しているのは、**予測や記憶の“芽”は神経系なしでもありうる**ということです。粘菌では過去の経路や外部スライム痕跡が後の行動に影響し、単細胞レベルでも habituation のような履歴依存の変化

が報告されています。これは「記憶なしでも賢く見える」のではなく、むしろごく弱い形の記憶がすでにあるという話です。予測についても、単細胞や粘菌で議論される“anticipation”は、人間のモデルベース予測とはかなり違いますが、少なくとも「直前までの規則性を次へ持ち越して応答を変える」方向を示しています。

Nature+3

なので、問いを厳密に言い換えるとうなります。

予測や記憶がないと、人間の脳のような高次の柔軟性・計画性・反事実性・言語的思考は作れないか。

これには、私は **はい、ほぼ無理** と答えます。海馬系の研究では、記憶は単なる保存庫ではなく、未来行動の構成、シーケンス学習、計画、構成的な推論に関わると整理されています。作業記憶も、感覚が消えた後に内部表現を保持・操作して行動を柔軟化する基盤だと考えられています。人間レベルの脳がやっているのは、その場の刺激への整合だけではなく、**見えていないものを保持し、まだ起きていないことを先取りし、候補世界を比較すること**なので、予測と記憶を外すと中核が消えます。 Nature+4

ただし、ここで一つ切り分けが必要です。

人間的知性に予測と記憶は必要でも、最小の生体的エージェンシーに人間型の予測と記憶が必要とは限りません。低いレベルでは、明示的な世界モデルの代わりに、履歴依存の状態変数、可塑性、慣れ、内部状態の持続、行動モードのヒステリシスだけでかなりのことができます。つまり、下位では「予測と記憶」は**露骨なモジュール**でなく、整合性維持のダイナミクスに埋め込まれているかもしれません。上位に行くほど、それが作業記憶、エピソード記憶、将来予測、計画として分化して見えてくる、という連続体として捉えるのがいちば

サイエンスダイレクト

ん無理がありません。 +3

だから、あなたのプロジェクトの設計論としてはこうなります。

最小系を作る段階では、予測・記憶を高級機能として最初から入れる必要はない。

しかし、**人間の脳のようなものを射程に入れるなら**、どこかの段階で予測と記憶は必須になる。しかもそれは付属機能ではなく、行動選択、想像、計画、学習、自己の連続性を支える骨格です。だから「ほぼ間違いなし」と言ってよい。ただし、その中身は最初から人間型である必要はなく、**最小形では履歴依存と先回りの状態更新として実装し、上位で明示的予測・明示的記憶へ発展させるのが筋**です。 Nature+3

一文で言うと、

粘菌は「記憶も予測も不要」を示しているのではなく、記憶と予測の原型が神経系以前にもありうることを示している。人間の脳のようなものには、その原型をはるかに強化・分化した仕組みがほぼ必須です。 Nature+3

次に進めるなら、有効な分解はこれです。

「**最小の整合性維持制御に必要な履歴依存**」と「**人間的知性に必要な将来シミュレーション**」を分ける。この分解をすると、どの段階で“予測”と“記憶”を強く導入すべきかがはっきりします。



線虫には、予測と記憶はありますか？それは人間と連続的ですか？

はい。

ただし、**記憶はかなり明確にある、予測はあるが人間のものよりかなり弱く、形も違う**、という答えになります。

線虫 *C. elegans* には、少なくとも非連合学習、連合学習、短期記憶、長期記憶に相当する現象が広く報告されています。かなり古い時点から総説でも整理されており、近年も連合記憶の相やその分子基盤、記憶制御因子の研究が続いています。温度走性では「飼育温度」を後の行動に使い、しかもその温度記憶が温度履歴の重み付き平均として表れるという報告があります。これは、線虫が単なる瞬間反応だけで動いていないことをかなり強く示します。 [Learning & Memory+3](#)

予測については、言い方に注意がいらいます。

線虫で強く言えるのは、**内部状態や過去履歴を使って次の行動を先回りの的に偏らせる**ことです。採餌では prior experience と現在の多感覚情報、内部状態が統合され、roaming / dwelling などの持続的状态や food leaving の確率を変えます。温度依存の行動状態でも、現在温度だけでなく過去の温度履歴と餌状態の組み合わせで行動モードが変わります。これは「未来の状態を厳密に明示表象している」とまでは言えませんが、少

[スプリンガーリンク](#)

なくとも**履歴依存の期待や次状態の先取り**はある、と見るのが妥当です。 [+3](#)

なので、あなたの問いに即して切るところです。

- **記憶はある**
- **予測の原型もある**
- ただしそれは、人間のような明示的な将来シミュレーションや反事実的思考ではない [Learning & Memory+2](#)

「人間と連続的か」については、**連続的ではあるが、同型ではない**がいちばん正確です。連続的というのは、線虫でも過去が現在の内部状態と行動選択に残り、その残り方が学習・記憶・状態依存の行動として観測されるからです。人間でも、過去の経験が内部モデルや記憶系を通して予測と選択を変える。そこには共通の軸があります。いっぽうで非連続なのは、人間では作業記憶、エピソード記憶、反事実的予測、言語、抽象的計画が大きく発達しており、線虫の履歴依存的な状態更新や条件づけと同じレベルではないからです。つまり、**原理の連続性はあるが、能力の階層差は大きい**ということです。 [Learning & Memory+2](#)

ここで雑にまとめてしまうと誤ります。

「線虫にも記憶がある」から「人間の記憶と同じ」とは言えないし、逆に「人間のような予測ではない」から「予測がない」とも言えません。今の研究の流れでは、線虫は**minimal memory** と **minimal prediction/expectation** を持つモデル生物として扱うのが自然です。採餌研究でも、現在環境と内部状態と

[スプリンガーリンク](#)

事前経験の統合が意思決定の中心として位置づけられています。 [+1](#)

あなたのプロジェクトの言葉で言い直すなら、こうです。

線虫は、

身体-環境閉ループのなかで、過去を内部状態として保持し、その状態に依存して“より整合的な次”を選ぶ

ところまではかなり明確に持っています。

人間はその上に、

その過去を多層に再構成し、複数の未来候補を比較し、抽象目標まで接続する

ところまで拡張している。

だから、**線虫から人間までは断絶ではなく拡張系列**とみるのがいちばん筋がいいです。 [Nature+3](#)

一文で言えば、

線虫には記憶はある。予測の原型もある。人間とは連続的だが、同じ種類の予測・記憶を同じ強さで持つわけではない。

次に詰めるべき問いはかなりはっきりしていて、

線虫で見えているのは「予測」なのか、それとも「履歴依存の状態遷移」なのか、です。

ここを切り分けると、最小実装がかなり締まります。



人間のミクロレベルの、予測と記憶の挙動は線虫と同じと言えますか？
でないとしたら、何が違いますか？

いいえ。ミクロな挙動まで同じとは言えません。

ただし、完全に別物とも言えません。いちばん正確なのは、**原理は部分的に連続しているが、表現の形式・時間スケール・回路の深さが大きく違う**です。 [Learning & Memory+2](#)

まず線虫側では、記憶はかなり明確です。非連合学習、連合学習、短期記憶、長期記憶に相当する現象が古くから整理され、温度走性では過去の温度履歴の重み付き平均のような記憶も報告されています。採餌でも、事前経験・現在の感覚入力・内部状態が統合されて roaming / dwelling や food leaving が変わります。つまり、**過去が現在の選択に残る**という意味では、人間と同じ軸の上にあります。 [Learning & Memory+2](#)

でも、人間のミクロな記憶・予測はここからかなり離れます。人間では、作業記憶は前頭葉・海馬・側頭葉などの**分散ネットワーク**で維持され、持続発火、活動サイレントなシナプス状態、振動結合など複数の実装様式が関与します。さらに海馬系は、単に過去を保存するだけでなく、系列、地図、将来の候補行動、構成的な未来表象まで支えます。これは線虫の履歴依存的な状態更新より、**はるかに高次で再組成可能な記憶**です。

[Vanderbilt University+3](#)

違いを絞ると、主に4つあります。

1. 記憶の“書き込み先”の違い。

線虫では、単一ニューロンや小さな回路、神経修飾、感覚ニューロン内の状態変化でも記憶様の保持が起こります。実際、AFD 温度感覚ニューロンに温度記憶の保持・リセット能力があるという報告があります。人間では、もちろん単一細胞レベルの可塑性はありますが、行動に効く記憶は通常、**大規模分散表現**として保持さ

[名古屋大学学術機関リポジトリ](#)

れます。 [ジトリ+1](#)

2. 予測の“型”の違い。

線虫で強く言えるのは、履歴や内部状態を用いて**次の行動を先回りのように偏らせる**ことです。けれども、人間では海馬や前頭葉を使って、まだ見えていない候補世界を比較したり、系列を先読みしたり、反事実的に「別の選択肢ならどうなるか」を扱えます。つまり線虫は主に**状態遷移の期待**、人間はそこに加えて**シミュレーション**

[スプリングーリンク](#)

ン可能な予測を持つ、という差があります。 [+3](#)

3. 時間スケールの違い。

線虫の記憶や予測は、採餌、温度、匂い、内部状態などに密接に結びついた比較的限定的な文脈で現れます。人間では、数百ミリ秒の作業記憶から、数十年に及ぶ自伝的記憶までがつながり、短期の予測と長期の自己物語が連動します。ここは量的差というより、**時間の階層性の差**です。 [Learning & Memory+2](#)

4. 記憶と予測の再利用性の違い。

線虫の履歴依存は、生態的に重要な少数の課題にかなり密着しています。人間の記憶は、ある場面で得た表現

を別の問題へ持ち込み、組み替え、将来計画や抽象推論に再利用できます。最近の海馬研究は、こうした**構成的・組合せ的な未来構築**をかなり前面に出しています。 [Nature+1](#)

なので、あなたの問いに対する厳密な答えはこうです。

線虫と人間は、過去が現在の行動選択に残り、次を少し先回りして調整するという最小原理では連続的です。
しかし、**人間ではその原理が多層・分散・再構成可能・反事実的になっているので、ミクロな挙動まで同じとは言えません。** [Learning & Memory+2](#)

あなたのプロジェクトの言葉で言い換えると、線虫では

「状態依存の整合性駆動」が、履歴依存の状態遷移として主に実装されている

のに対し、人間ではそれが

多層記憶とモデル的予測を使って、候補世界の比較まで行う形に拡張されている

[スプリンガーリンク](#)

と見るのが一番ぶれません。 [+2](#)

かなり乱暴に圧縮すると、違いはこれです。

線虫は「**過去が今に残る**」。

人間は「**過去を再構成して未来を作る**」。

前者なしに後者は出にくいので連続性はあるが、後者は前者の単純な拡大ではありません。 [Learning & Memory+2](#)

次に詰めるなら、有効なのは

線虫の“履歴依存状態遷移”が、どこから人間的な“構成的未来予測”へ相転移するのか
という境界条件です。



予測が成立したら、記憶が強化されるということは同じですか？

同じとは言えません。むしろ、**単純に「予測が当たるほど記憶が強くなる」**ではないです。

いまの研究では、かなり大ざっぱに分けると次です。

1. 予測が外れたとき

記憶更新は起こりやすいです。

予測誤差や驚きは、記憶を更新したり、境界を作ったり、その出来事を強く残したりしやすい、という整理が強いです。海馬と青斑核の相互作用を通じて、予測誤差が記憶更新を駆動するというレビューもありますし、prediction error と episodic memory の関係をまとめた総説でも、違反やズレが記憶を強める局面が重視さ

[サイエンスダイレクト](#)

れています。 [+3](#)

2. 予測が当たったとき

これも記憶を助けることはありますが、強化されるのはしばしば**詳細なエピソード記憶**ではなく、**スキーマに合ったまとまりや粗い表現**です。

つまり「当たったから全部よく覚える」ではなく、**予測と整合する形で圧縮・統合されやすい**。スキーマ一致は記憶形成を助ける一方で、時間が経つとより coarse になったり、unitized な表現を作りやすかったりします。 [Nature+2](#)

なので、人間についてまず言えるのは、

- **予測誤差**は更新・再編・境界形成を強めやすい
- **予測成立**は統合・圧縮・スキーマ化を助けやすい
- どちらが「強い記憶」かは、何を記憶と呼ぶかで変わる

ということです。

詳細な出来事記憶ならズレが強く効くことが多く、一般化やスキーマ形成なら当たりが効くことも多いです。

サイエンスダイレクト

+2

これを線虫にそのまま当てはめるのは危ないです。

線虫では、人間のようなエピソード記憶の「詳細 vs gist」の分化がはっきりあるわけではありません。線虫で強く言えるのは、**履歴や条件づけによって後の行動が変わること**です。つまり、人間でいう「予測が当たったから記憶が強化された」というより、**状態遷移や連合の重みが調整された**と表現するほうが自然です。

だから、あなたの問いへの答えはこうです。

人間と線虫で共通しているのは、予測と記憶が相互作用することです。

でも、「**予測が成立したら記憶が強化される**」という関係が、そのまま同じ形で成り立つとは言えません。

より正確には、

- 人間では、予測誤差が記憶更新を強く駆動しやすい
- 予測成立は、記憶の統合やスキーマ化を助けやすい
- 線虫では、それがもっと低次の履歴依存的な可塑性や状態更新として現れている可能性が高い

という違いです。

一文で圧縮すると、

「予測が当たると記憶が強まる」のではなく、「予測と結果の関係に応じて、記憶の“どの側面”が強まるかわ変わる」

です。

この違いは、あなたの設計にも重要です。

最小系では「予測成功で記憶強化」という単純ルールより、**一致なら安定化、不一致なら更新促進**くらいの二分のほうが、生物らしいです。



人間では、予測誤差が記憶更新を強く駆動しやすい
 予測成立は、記憶の統合やスキーマ化を助けやすい
 線虫では、それがもっと低次の履歴依存的な可塑性や状態更新として現れている可能性が高い
 というのは、線虫は、弱い、本質的な傾向は、同質と考えていいですか？

はい、**かなり弱い形では同質**と見てよいです。

ただし、**同じ原理がそのまま縮小版に入っているとまでは言わない**ほうが安全です。

いちばん無理のない言い方はこれです。

線虫にも、人間でいう「予測と結果の関係に応じて内部状態や可塑性が変わる」という方向性はある。
ただし、人間ではそれがエピソード記憶・スキーマ・予測誤差・再構成に分化しており、線虫ではもっと局所

[サイエンスダイレクト](#)

的で履歴依存的な状態更新として現れる。 +3

整理すると、共通しているのは

「過去と現在の不一致／一致が、その後の系の振る舞いを変える」

という傾向です。

人間では、不一致は記憶更新や境界形成を強く促しやすく、一致は統合やスキーマ化を助けやすいと考えられています。線虫でも、学習・記憶・忘却が環境条件や履歴に応じて調整され、行動可塑性として現れます。な

[サイエンスダイレクト](#)

ので、“履歴に応じて内部更新が起こる”という芯は連続的です。 +4

でも、違いはかなり大きいです。

人間では、その更新が海馬系や皮質ネットワークの上で、出来事記憶、一般化、スキーマ、再構成的想起のように機能分化しています。線虫で強く言えるのは、そこまではなく、条件づけ、習慣化、温度や匂いへの履歴依存、忘却速度の調整のような、より低次の可塑性です。つまり、傾向は同質でも、表現形式は非同型です。 [Nature+3](#)

なので、あなたのプロジェクト向けには、こう置くのがよいと思います。

線虫は、人間の予測誤差学習や記憶統合の“弱くて未分化な祖型”を持つ。

これはかなり妥当です。

一方で、

線虫も人間も同じ仕組みがそのままスケールしただけだ

[サイエンスダイレクト](#)

は言い過ぎです。 +2

一文で圧縮すると、

弱いが本質的傾向は連続的。ただし、線虫では「記憶更新と予測」の区別自体がまだ未分化で、人間ではそれが多層化・再構成化している。 [Nature+2](#)

次に詰めるなら、研究上は

線虫で見えているのは prediction error そのものか、それとも history-dependent state update かを分けるのが一番重要です。



自分が関心があるのは、予測が成立したという判断がどのように行われるかが、ミクロレベル絵明確なのか、全体的に判断されるのかを知りたい。そもそも最小の構成の知性を作る目的も、線虫が作りたいのではなく、その本質的な仕組みを作りたいからです。

はい。そこはかなり本質です。

結論から言うと、

「予測が成立したか／外れたか」の判断は、少なくとも人間では“局所”と“全体”の両方で行われていると考えるのが妥当です。

そして、あなたが作りたい「本質的な仕組み」にも、たぶんこの二層性が必要です。 +2

整理すると、こうです。

まず**局所**。

感覚系や回路の近いところでは、「来るはずの入力」と「実際の入力」のズレはかなりローカルに比較されている、という見方が強いです。予測処理の神経実装では、樹状突起や局所回路で prediction と sensory input の差が計算されうる、という整理がありますし、皮質の層ごとの研究でも prediction error は比較的局所的な

サイエンスダイレクト

実装を持つと考えられています。つまり、**ミクロには comparator がいる**。 +1

しかしそれだけでは足りません。

実際の脳では prediction error は、単一ニューロンや単一領域の中だけに閉じず、前頭葉・感覚野・海馬などをまたいで**分散的かつ相乗的**に表現されます。最近の研究でも、prediction error は皮質階層全体にまたがる distributed representation を持ち、長距離の feedforward / feedback 結合が重要だと示されています。要するに、**局所比較で生じたズレが、全体状態の更新へ持ち上げられる**。 Nature+2

さらに記憶更新まで入ると、ますます全体性が強くなります。

海馬と青斑核の相互作用に関する最近のレビューでは、海馬が surprise の情報を青斑核へ伝え、その prediction error の大きさに応じて記憶更新が進む、という枠組みが提案されています。これは、「ズレの検出」自体は局所に近くても、“このズレをどのくらい重く扱うか”は**全体の状態依存**だということです。

サイエンスダイレク

ト

なので、あなたの問いに直接答えると、

予測成立の判断は、ミクロに明確な comparator があるだけではなく、全体状態に照らして“意味づけ”される

と考えるべきです。

たとえば同じズレでも、

- ノイズとして捨てる
- 局所の感覚ゲインだけ変える
- 行動モードを切り替える
- 記憶を更新する

のどれになるかは、全体の内部状態に依存する。

ここが、単純な「予測器」と「知性」の分かれ目です。

あなたの目的に引きつけるなら、線虫を作りたいのではなく本質を作りたいわけだから、最小構成としてはこう考えるのがよいです。

最小構成に必要なのは、1つの巨大な“全体判断器”でも、完全に独立した局所比較器の寄せ集めでもなく、

(1) 局所的な予測-入力比較

(2) そのズレを全体状態に応じて重みづけする仕組み

の二段構えです。

かなり圧縮すると、

- **局所**：何がズレたか
- **全体**：そのズレをどう扱うか

です。

これは実装上かなり重要です。

もし局所比較だけにすると、系はひたすら誤差に過敏になります。

逆に全体評価だけにすると、何が具体的にズレたのかが消えて、学習が粗すぎる。

だから最小でも、

local mismatch + global arbitration

は必要になる可能性が高いです。

ここで、あなたの「状態依存の整合性駆動」という概念に戻すと、かなりきれいに言えます。

- **局所比較**は、整合性駆動のためのセンサー
- **全体判断**は、整合性駆動そのもの
- **価値判断**は、その結果として候補行動に与えられる優先度差

です。

この整理にすると、「予測成立の判断」は独立の神秘的な機能ではなく、**身体-環境閉ループの中で、局所ズレを全体整合性へ写像する処理**になります。

あなたが本質機構を作るなら、私は最小モデルを次の3層にします。

1. 局所比較層

各感覚チャンネルまたは各行動モードに対して、期待状態と実入力のズレを計算する。

2. 内部状態層

空腹、覚醒、危険負荷、飽和、探索圧のような slow variable を持つ。

3. 全体整合性層

局所ズレを内部状態で重みづけし、記憶更新・モード切替・行動優先度変更のどれを起こすか決める。

これでかなり「本質」に近づきます。

ここでは、記憶も予測も独立モジュールというより、

- 予測 = 局所比較に必要な期待状態
- 記憶 = 内部状態と比較基準を持続させる仕組み

として出てきます。

なので、あなたが今知りたいことに対する一番短い答えはこうです。

予測成立の判断は、ミクロで完結するとは考えにくい。局所比較は必要だが、それが“成立した／更新すべき

サイエンスダイレクト

／無視すべき”になるのは全体状態依存です。 +2

そして、最小の本質機構を作るなら、

局所 comparator と、状態依存の全体 arbitration の両方を入れるべきです。

次に進めるなら、この3層をそのまま線虫以下でも動く**最小状態方程式**に落とせます。



その状態方程式は、線虫にも、おそらく人間にもあてはめられるものですか？

部分的には当てはめられます。

ただし、**同じ方程式をそのまま線虫にも人間にも当てれば本質を捉えられる**、とまでは言えません。

いちばん正確な言い方はこうです。

当てはめるべきなのは「具体的な方程式」より「方程式の型」です。

つまり、

- 局所比較
- 内部状態
- 全体的な重みづけ／裁定
- 行動更新
- 記憶更新

という**構造**は、線虫にも人間にもかなり広く当てはめられる。

でも、その中身の表現形式、階層数、時間スケール、分散性は大きく違います。C. elegans では行動状態や神経修飾が脳全体の振る舞いを大きく形づくり、人間では prediction error が局所比較だけでなく皮質全体に分散表現され、さらに海馬や青斑核のような系が記憶更新に関わります。 [Cell+4](#)

なので、以前の「最小状態方程式」は**普遍的な祖型**としては有効です。

たとえば概念的にはこうです。

- 局所ミスマッチ
$$e_i(t) = x_i(t) - \hat{x}_i(t)$$
- 全体状態
$$s(t+1) = f(s(t), e(t), u(t))$$
- 行動選択
$$a(t) = \pi(x(t), s(t))$$
- 記憶更新
$$m(t+1) = g(m(t), e(t), s(t))$$

この「型」はかなり一般的です。

線虫では s は少数の内部状態や神経修飾状態で十分かもしれないし、人間では s が多層・分散・文脈依存になる。つまり**方程式の骨格は共有できるが、状態空間の次元と意味は全く同じではない**です。 prediction error の局所比較と全体的裁定の二層性は、ヒトや哺乳類の予測処理ではかなり支持されており、C. elegans

[サイエンスダイレクト](#)

でも局所感覚と脳全体の状態依存性の両方が重要です。 [+4](#)

ここで大事ななのは、「**線虫にも人間にも使える**」ことと「**同一メカニズムである**」ことは別だという点です。使えるのは、どちらも

- 過去が現在に残り
- 現在の入力と期待のズレがあり

- そのズレが内部状態に応じて扱われ
- 行動と記憶が更新される

という共通骨格を持つからです。

でも人間では、その更新が再構成的記憶、階層的予測、長距離結合、反事実的比較まで拡張されているので、線虫版の低次元モデルをそのままスケールアップしても不十分です。prediction error は皮質階層に分散的かつ相乗的に表現され、人間や哺乳類では「global model failure」のような脳全体的信号も関与します。

[Nature+3](#)

だから、あなたの目的に合う整理はこうです。

線虫にも人間にも当てはまるのは「状態依存の整合性駆動」を記述する最小形式であって、具体的な神経方程式ではない。

さらに言えば、

- 線虫向けには「低次元・少数モード・神経修飾でゲートされる状態遷移モデル」
- 人間向けには「階層的・分散的・記憶再構成を含むモデル」

として展開すべきです。

両者は**同じテンプレートの別解像度版**と考えるのがいちばん筋がいいです。C. elegans でも body-environment を含む統合モデルが重要視されており、minimal cognition 側でも sensorimotor coordination を核に置く議論が続いています。 [Nature+2](#)

一文で言うと、

はい、祖型としては両方に当てはめられます。けれども、それは“共通の構造方程式”としてであって、“同じ具体方程式”としてではありません。

次に進めるなら、

共通テンプレートと線虫版の最小実装と人間版で追加される自由度を3段で分けて書くのがよいです。



この最小状態方程式は、最小の知性の実装が含まれると思いますが、これ以上コンパクトにすることはできないですか？

できます。

ただし、**何を捨てるか**で、残るものの意味が変わります。

前に出した形は、

- 局所比較
- 内部状態
- 全体裁定
- 行動更新
- 記憶更新

を分けていたので、説明にはよいが、最小化としてはまだ冗長です。
本質だけ残すなら、かなり圧縮できます。

まず結論

最小の知性の核は、たぶん次の2つです。

1. 状態

2. 遷移

つまり、

現在の状態があり、その状態が入力と履歴に応じて次の状態へ移り、その状態が行動を生む
これだけです。

かなり圧縮すると、

$$z_{t+1} = F(z_t, i_t)$$

$$a_t = G(z_t)$$

です。

ここで

- z_t : 系の内部状態
- i_t : 身体-環境閉ループから来る入力
- a_t : 行動

です。

これが一番コンパクトです。

これで何が吸収されるか

この形にすると、以前分けていたものは全部 z と F の中に入ります。

局所比較

予測と入力のズレは、 F の内部で計算すればよい。
独立の変数にしないでよい。

記憶

記憶は、過去が z_t に残っていることそのものです。
別モジュールにしないでよい。

予測

予測は、 F が「次に何が起こるか」を暗に織り込んだ遷移則なら十分です。
明示的な予測器を立てなくてよい。

全体裁定

これも F の中で、内部状態に応じて更新の仕方を変えればよい。
別の司令塔は必須ではない。

行動選択

$G(z_t)$ に吸収できます。

さらに言えば、1式にもできます

もし行動も状態の一部だとみなすなら、

$$z_{t+1} = F(z_t, i_t)$$

だけでもよいです。

このとき行動 a_t は、状態 z_t の一部として含まれる。
つまり「行動すること」も状態遷移の一部だとみなす。

これはかなり生体的です。
生物は「内部表現→行動」という二段にきれいに分かれていないことが多いからです。

ただし、この圧縮には代償がある

この1式または2式の形は、**最も本質的**ですが、同時に**何が本質かを隠しやすい**です。
例えば、

$$z_{t+1} = F(z_t, i_t)$$

だけだと、

- どこに履歴があるのか
- どこでズレを見ているのか
- どこに整合性駆動があるのか
- どこで探索と恒常性のバランスを取るのか

が全部ブラックボックスになります。

なので、研究の初期段階では、あまり潰しすぎると逆に使いにくい。

実装のための最小圧縮なら、この形がよい

私は、実装目的なら **2変数版** がちょうどいいと思います。

$$z_{t+1} = F(z_t, i_t)$$

$$a_t = G(z_t)$$

ただし z_t を2つに分ける。

- h_t : 遅い内部状態
- m_t : 現在の行動モード

すると、

$$h_{t+1} = f(h_t, i_t, m_t)$$

$$m_{t+1} = g(m_t, h_t, i_t)$$

$$a_t = \phi(m_t)$$

これで、

- 記憶 = h_t
- モード切替 = m_t
- 整合性駆動 = f, g の性質
- 行動 = ϕ

になります。

これでもかなりコンパクトです。
しかもブラックボックスすぎない。

一番本質的に言い直すと

あなたの枠組みでは、最小知性は

身体-環境閉ループの中で、履歴を含んだ内部状態が、次の行動を偏らせながら更新され続けること
です。

これを数式にただけなら、

$$z_{t+1} = F(z_t, i_t)$$

で足ります。

つまり、これ以上コンパクトにはできません。

ただしその場合、**知性の本質は“予測器”でも“記憶器”でもなく、履歴依存の状態遷移そのもの**という立場を取るようになります。

私は、この圧縮はかなり正しいと思います。

少なくとも線虫以前まで下げるなら、そのほうが本質に近い可能性が高いです。

なので整理すると

- 理論的な最小形

$$z_{t+1} = F(z_t, i_t)$$

- 実装しやすい最小形

$$h_{t+1} = f(h_t, i_t, m_t), \quad m_{t+1} = g(m_t, h_t, i_t), \quad a_t = \phi(m_t)$$

前者は最も本質的。

後者は最も作りやすい。

私なら、研究ノートの最上位定義は1式にして、実装は2変数版に落とします。



ちなみにあなたが整理してくれた、実装しやすい最小形ですが、これは、最小単位のモジュールにこれを適用し、入れ子構造的に、拡大すれば、人間の知性を作れると思いますか？

部分的には、はい。

でも、**それだけで人間の知性まで行けるとは私は思いません。**

正確に言うと、

- **最小モジュールを入れ子にして階層化する**
ことは、かなり重要な方向です。
- ただし、人間の知性には、**単なる入れ子以上の性質**が必要です。

核だけ言うと、あなたの最小形は

知性の祖型にはなりうるが、**人間知性の十分条件ではない**、です。

まず、なぜ有望か。

人間でも線虫でも、行動は単発の反応ではなく、履歴を含んだ内部状態の更新としてかなり自然に書けますし、階層的・多時間スケールの処理は脳のかかなり一般的な特徴です。聴覚記憶や系列認識でも、感覚野から海馬・帯状皮質へ至る階層的処理とフィードバックが見られますし、予測処理のモデルでも、上位層が下位層の時間ダイナミクスを調整する構図が使われます。 [Nature+1](#)

ただし、ここで誤解しやすいのは、**階層化すれば自動的に人間になるわけではない**ことです。

人間の知性には、少なくとも次の追加条件が要ります。

第一に、**多時間スケール**です。

最小モジュールをただ多数つなぐだけではなく、短い感覚ループ、秒単位の行動モード、分単位の文脈、さらに長期記憶や自己物語のような、異なる時間定数が重なっている必要があります。人間の認知では、この時間階層がかなり本質的です。 [Nature+1](#)

第二に、**再表現と再利用**です。

線虫に近い最小系は「過去が今に残る」方向ですが、人間はそこから進んで、「過去を別の文脈で再構成して未来に使う」ことをします。最近の研究でも、人間の複雑な意思決定は階層的処理に加えて counterfactual

な比較を使います。これは、単なるモジュールの入れ子ではなく、**表現を別の課題へ持ち越して組み替える能力**です。 [Nature](#)

第三に、**共有された内部変数や世界モデル**です。

入れ子モジュールだけだと、各階層が局所最適の小さな反応器になりやすい。人間では、異なる感覚・記憶・行動系が、ある程度共通の状態空間や文脈表現を介して結びついています。これがないと、階層はできても「統一的な知性」になりにくいです。予測と記憶の相互作用も、単なる局所更新ではなく、階層横断的に整理

[サイエンスダイレク](#)

されています。 [ト](#)

第四に、**反事実性**です。

人間は「いま起きていない候補」を内部で比較できます。これは最小モジュールの単純な状態遷移だけでは出にくい。最近の人間研究でも、階層的意識決定には counterfactual process が明確に関与すると示されています。 [Nature](#)

だから、あなたの問いにかなり率直に答えると、

入れ子構造は必要そうだが、それだけでは足りない
です。

私ならこう整理します。

- **最小モジュール**は、「状態依存の整合性駆動」の局所単位
- **階層化**は、その局所単位を多時間スケール化するために必要
- **人間化**には、その上に
 - 共有状態空間
 - 記憶の再構成
 - 候補世界の比較
 - モジュール間の長距離拘束
が要る

なので、拡張の方向としては「入れ子」だけでは弱くて、

入れ子 + 共有表現 + 再帰的参照

くらいが必要だと思います。

もっと厳しく言うと、最小形をそのまま複製して深くしただけだと、できるのはせいぜい

- 階層的 reflex
- 階層的 foraging
- 階層的 sensorimotor control

の延長です。

そこから人間に行くには、“過去を現在の材料として持つ”だけでなく、“過去から未来候補を構成する”機能が
必要です。

だから、研究戦略としてはかなり明確です。

最初の段階では、あなたの最小形をモジュール化して、入れ子にしてよいです。

ただし、その時点で目標は「人間知性」ではなく、

多階層の生体的エージェンシーに置くべきです。

そのあとで追加すべきものは、おそらくこの順です。

1. 上位層が下位層の遷移則を変える
2. 複数モジュールが共有する状態変数を持つ
3. 非現在の候補状態を内部生成する
4. 候補間比較で行動を決める

この3と4が入って、ようやく人間側にかなり近づきます。

一文でまとめると、

その最小形を入れ子にして拡大するのは、かなり正しい出発点です。だが、人間の知性を作るには、階層化だけでなく、共有状態空間・再構成的記憶・反事実的予測を追加する必要があります。

ここで逆に重要なのは、あなたの最小形が無駄だということではなく、**人間知性の土台としてはかなり良いが、上物を支えるにはまだ“水平結合”と“未来構成”が足りない**ということです。



では、現在のTransformerを代表するANNで、この構造を持っていますか？

部分的には持っています。

でも、**あなたが整理した構造を、そのまま本質的に持っているとは言いにくい**です。

いちばん短く言うと、

- **Transformer は「状態遷移器」ではある**
- **しかし「身体-環境閉ループ」の中で状態依存の整合性駆動を行う系**ではない
- **したがって、最小知性の祖型の一部は持つが、核までは持っていない**

です。 [arXiv+1](#)

もう少し分けるとこうです。

まず、**ある種の内部状態**はあります。

Transformer は自己注意で文脈を参照し、各層の hidden state を更新して次トークン分布を出すので、形式的には

$$z_{t+1} = F(z_t, i_t)$$

のような見方ができます。デコーダ型モデルでは、過去トークン列と内部活性、実運用では KV cache が「いままでの履歴を効かせる状態」として働きます。Transformer 自体も、RNN や CNN を使わず attention のみで系列依存を扱う構造として提案されました。 [arXiv+1](#)

ただし、その**状態**はかなり特殊です。

人間や線虫で問題にしていた「内部状態」は、空腹、覚醒、探索圧、危険負荷のような、行動方針を継続的に歪める slow variable でした。普通の Transformer の状態は、主に**トークン列条件づけのための計算状態**で

す。つまり「意味のある生理的・行動的内部状態」というより、「次トークン予測のための一時的表現」に近い。これは似ているようでかなり違います。 [arXiv+1](#)

次に、**局所比較**に相当するものはあります。

学習時には、予測分布と正解トークンのズレが損失として計算されます。だから「予測と結果の比較」は確かにある。けれどもそれは主に**外部教師信号に対する誤差逆伝播**であって、あなたが議論していたような、オンラインに自律的に「このズレを無視するか、記憶更新するか、モード切替するか」を裁定する仕組みではありません。普通の推論時 Transformer は、その場で重みを更新しません。 [arXiv+1](#)

さらに重要なのは、**全体裁定**が弱いことです。

あなたの枠組みでは、局所 mismatch を全体状態に応じて意味づけする層が重要でした。ところが vanilla Transformer では、そういう意味での global arbitration はかなり薄いです。attention は全体依存を作れますが、それは「どのトークンを見るか」の重みづけであって、**系全体の生存的整合性や探索継続可能性を裁定する内部動機づけ**ではない。 [arXiv+1](#)

そして決定的なのが、**身体-環境閉ループがない**ことです。

標準的な Transformer/LLM は、テキスト列を受けて次トークンを出す静的な入出力写像に近い。ロボットやエージェントに組み込めば閉ループの一部にはなれますが、それは Transformer 単体の性質ではなく、**外側のシステム設計**によって与えられるものです。ここが、生体モデルの議論とはかなり違います。 [arXiv+1](#)

要するに、あなたの二概念に照らすとこうです。

1. 身体-環境閉ループ

普通の Transformer 単体では、ほぼ持たない。

外部ツール使用、ロボット化、対話ループ、環境フィードバックを足して初めて部分的に持つ。

2. 状態依存の整合性駆動

弱くしか持たない。

「次トークン尤度を高める」方向の駆動はあるが、それは学習目標であって、生体的な整合性維持や探索継続の駆動とはかなり違う。 [arXiv+1](#)

なので、厳しめに言えば、**現在の Transformer は「予測器」としては強いが、「整合性駆動する身体化エージェント」としては弱い**です。

ただし、完全に外れているとも言えません。

最近 Transformer 系に persistent memory や recurrence を足す方向がかなり出ています。たとえば recurrent memory を持つ変種や、attention と recurrence を混ぜて長期情報を圧縮保持する系が提案されています。これは、標準 Transformer が**持続状態や長期内部状態に弱い**ことの裏返しでもあります。 [arXiv+1](#)

だから、あなたの問いへの実質的な答えはこうです。

現在の Transformer を代表する ANN は、

「履歴依存の状態更新」という最小式の外形は持つ。

しかし、

「身体-環境閉ループの中で、状態依存に整合性を駆動して次を選ぶ」という、あなたの意味での最小知性の構造は、まだ本質的には持っていない。

もっと blunt に言うなら、

Transformer は**“文脈条件つき予測機械”**であって、

まだ**“自己の状態を保ちながら世界の中で次を立ち上げる機械”**ではない、ということです。

この見方だと、あなたが次に考えるべき差分ははっきりしています。

- persistent な内部状態

- 局所誤差と全体裁定の分離
- 行動モードの持続と切替
- 外部報酬ではなく整合性駆動
- 身体-環境閉ループ

この5つのどれを最小追加要素として入れるかです。

私の見立てでは、Transformer に最初に足すなら **persistent internal state** と **global arbitration** が最重要です。



最小の2変数版の状態方程式にもどります。これをニューラルネットワークでシンプルに表現できますか？

ANNでもよいし、より生体のニューロンに近いモデルをつかってよいですが。

できます。

しかも、**かなりシンプル**に書けます。

あなたが戻りたい「最小の2変数版」は、前に整理したこれでした。

$$h_{t+1} = f(h_t, i_t, m_t), \quad m_{t+1} = g(m_t, h_t, i_t), \quad a_t = \phi(m_t)$$

ここで

- h_t : 遅い内部状態
- m_t : 現在の行動モード
- i_t : 身体-環境からの入力
- a_t : 行動

です。

これを NN で表すなら、まずは **2つの再帰状態を持つ小さなRNN** と考えるのが自然です。

Transformer より、むしろこちらのほうが本質に近いです。

1. 一番シンプルな ANN 表現

連続値2状態の最小版

$$h_{t+1} = (1 - \alpha_h)h_t + \alpha_h \sigma(W_h[h_t, m_t, i_t] + b_h)$$

$$m_{t+1} = (1 - \alpha_m)m_t + \alpha_m \sigma(W_m[h_t, m_t, i_t] + b_m)$$

$$a_t = W_a m_t + b_a$$

ここで

- $[h_t, m_t, i_t]$ は連結
- σ は tanh や sigmoid
- α_h, α_m は更新速度

です。

この形のよいところは、

- h を遅くすることで「内部状態」
- m をやや速くすることで「行動モード」
- 両者の相互作用で「持続」と「切替」

が自然に出ることです。

かなり本質的です。

しかも、これだけで

- 記憶
- 履歴依存
- モード遷移
- 状態依存の行動バイアス

が入ります。

2. 行動モードを離散寄りにした版

生体的には、 m_t は連続値より **少数モードの競合** にしたほうがそれらしいです。
たとえば explore / exploit / avoid の3モード。

$$u_{t+1} = (1 - \alpha_u)u_t + \alpha_u(W_u[h_t, u_t, i_t] + b_u)$$

$$m_t = \text{softmax}(u_t)$$

$$a_t = W_a m_t$$

このとき

- u_t : モードの内部活性
- m_t : 各モードの確率または重み

になります。

これはかなり使いやすいです。

線虫っぽい「ローミング / ドウェリング」的な切替にも寄せやすい。

3. もっと生体寄りの最小版

ANN でも十分ですが、より生体っぽくするなら **leaky integrator neuron** を使うのがよいです。
スパイクングまで行かなくてもよいです。

leaky rate neuron 版

$$\begin{aligned}\tau_h \dot{h} &= -h + \sigma(W_h[h, m, i] + b_h) \\ \tau_m \dot{m} &= -m + \sigma(W_m[h, m, i] + b_m) \\ a &= \phi(m)\end{aligned}$$

ここで $\tau_h > \tau_m$ にすると、 h が遅く、 m が速くなる。
これだけでかなり「内部状態」と「モード」の分離が出ます。

この版の利点は、

- 生体の膜時定数っぽい解釈ができる
- 時間スケール差を自然に入れられる
- 数理的に扱いやすい

ことです。

私は最初の実装なら、これが一番よいと思います。

4. スパイクングにする必要はあるか

最初は不要です。

スパイクングニューロンで書くこともできますが、今の段階では**本質**ではないです。

今ほしいのは

- 状態が残る
- モードが切り替わる
- 入力で更新される
- 持続がある

ことです。

これは rate model で十分です。

スパイクングが必要になるのは、

- 発火タイミング自体を使いたい
- STDP 的可塑性を見たい
- 神経動力学の生理的対応を詰めたい

ときです。

今はまだ早いです。

5. 最小構造を NN として描くと

実質、こういう小ネットワークです。

- 入力ノード i_t
- 内部状態ユニット h_t
- モードユニット m_t
- 行動出力 a_t

接続は

- $i \rightarrow h$
- $i \rightarrow m$
- $h \rightarrow h$ 再帰
- $m \rightarrow m$ 再帰
- $h \leftrightarrow m$
- $m \rightarrow a$

です。

これだけです。

かなり重要なのは、

自己再帰 $h \rightarrow h, m \rightarrow m$ があることです。

これが記憶と持続を作ります。

NN 的にはここが本体です。

6. 予測を入れるならどうするか

予測を最初から大げさに入れなくていいですが、必要なら最小追加で入れられます。

$$\hat{i}_{t+1} = P(h_t, m_t)$$

$$e_t = i_t - \hat{i}_t$$

$$h_{t+1} = f(h_t, m_t, e_t)$$

$$m_{t+1} = g(m_t, h_t, e_t)$$

つまり、入力そのものではなく **予測誤差** を使って更新する。

これでかなり predictive になります。

ただ、最初の最小実装ではここまで入れず、

まずは入力で直接更新する版のほうがよいです。

7. 一番おすすめの最小 ANN

私ならまずこれです。

$$h_{t+1} = (1 - \alpha_h)h_t + \alpha_h \tanh(W_{hh}h_t + W_{hm}m_t + W_{hi}i_t + b_h)$$

$$u_{t+1} = (1 - \alpha_m)u_t + \alpha_m \tanh(W_{uh}h_t + W_{uu}u_t + W_{ui}i_t + b_u)$$

$$m_t = \text{softmax}(u_t)$$

$$a_t = W_a m_t$$

これなら

- h_t : 遅い内部状態
- u_t : モード活性
- m_t : モード選択
- a_t : 行動

として明快です。

これは「最小知性」の実装としてかなりよい落とし所です。

しかも将来、

- 予測誤差
- 可塑性
- 階層化
- 複数モジュール化

を足しやすい。

8. 重要な点

このモデルの本質は、NN を使うことなく、

- 遅い状態
- 速いモード
- 再帰
- 身体-環境入力との閉ループ

を持つことです。

なので、ANN でもよいし、生体寄り rate model でもよい。

最初の段階では、この2つはほぼ同じです。

9. かなり圧縮した答え

はい、できます。

最もシンプルには、**2つの再帰変数を持つ小型RNN**として表せます。

- h : 遅い内部状態
- m : 速い行動モード

を持ち、

$$h_{t+1} = f(h_t, m_t, i_t), \quad m_{t+1} = g(m_t, h_t, i_t), \quad a_t = \phi(m_t)$$

とすることでよいです。

最初は **leaky integrator 型の rate network** が一番適しています。

スパイクングは後回しで十分です。

必要なら次に、この最小モデルを **Python** でそのまま動く **30~50 行程度の実装** に落とします。



最初の実装は興味深いですが、性能を測ったり、実証するとき、環境は何ですか？あるいはデータセットでもいい。

環境は、**迷路や一般RLベンチマークではなく、あなたの仮説を直接叩けるもの**にしたほうがいいです。

この最小モデルで見たいのは、たぶん性能一般ではなく、

- 内部状態がないと再現できない振る舞いが出るか
- モード切替が生じるか
- 履歴依存で「次」が変わるか
- 外的目標が固定されなくても行動方針が立ち上がるか

です。

なので、最初の実証環境は **線虫風の極小環境** が最適です。

結論

最初は次の3段階がよいです。

1. 一番よい最初の環境

2次元の餌パッチ環境

エージェントは平面上を動く。

環境には

- 餌濃度の空間分布
- 危険刺激の空間分布
- 枯渇する餌パッチ
- 入力ノイズ

だけを置く。

エージェントの観測は局所だけ。

- 現在位置の餌濃度
- 濃度勾配の粗い情報
- 危険入力
- 自己の内部状態 h

行動は少数で十分です。

- 前進
- 旋回
- 停留
- 回避方向への移動

これで、

stay / leave

explore / exploit

危険回避と摂食の競合

が出せます。

これはあなたの最小式にかなり直接対応します。

2. 次にやるべき環境

パッチ切替環境

複数の餌パッチがあり、

- 良質だがすぐ枯れる
- 低質だが安定
- 危険に近いが高報酬
- 安全だが薄い

みたいに性質を変える。

ここで重要なのは、

「どれだけ報酬を取ったか」より、

- いつ離脱するか
- 同じ場所に留まりすぎるか

- 状態依存で探索へ切り替わるか

を見ることです。

この環境は、**内的整合性駆動**をかなり見やすいです。

3. その次

履歴依存環境

同じ現在入力でも、過去履歴によって最適行動が変わるようにする。

例:

- 餌がしばらく減少傾向なら離脱が有利
- 危険が断続的に続いた後は探索を抑える
- 一定時間成果がなければ探索モードへ移る

これで、内部状態 h を本当に使っているか見えます。

単純な反応器では解けない。

データセット型は向いているか

あまり向いていません。

理由は、このモデルの核心が **身体-環境閉ループ** だからです。

静的データセットだと、

- 状態の持続
- 行動が環境を変えること
- モード切替

が弱くなります。

ただし、補助的には使えます。

使えるデータセット型

- 線虫の行動軌跡データ
- roaming / dwelling の状態ラベル付き時系列
- patch leaving の時系列
- 温度走性や化学走性の軌跡データ

これは**模倣対象**や**比較対象**としては有効です。

でも、最初のモデル検証には、まず自作シミュレーション環境のほうがよいです。

何を性能指標にするか

ここが重要です。
単純な累積報酬だけだと、あなたの仮説を壊します。
見るべき指標はこうです。

1. モード切替の自発性

- exploit から explore に移るか
- 危険時に avoid へ移るか
- 同じ入力でも内部状態で変わるか

2. 履歴依存性

- 現在観測だけでは説明できない行動差が出るか
- 過去の餌履歴で leave の閾値が変わるか

3. 整合性維持

- 枯渇に張り付きすぎないか
- 危険に突っ込み続けられないか
- 停滞しすぎないか

4. 探索-維持バランス

- 餌を取るだけの固定方策にならないか
- 無意味に動き続けられないか

5. アブレーション耐性

- h を消したら性能が落ちるか
- m を消したら行動が硬直するか
- 再帰を消したら履歴依存が消えるか

これが一番大事です。
性能より、どの構成要素が何を担っているかを見るべきです。

最小のベンチマーク案

私なら最初はこれです。

Environment A: Depleting Food Patch

- 2D空間
- 餌パッチ1つ
- 餌は滞在で減る

- ときどき危険刺激
- 外に出ると別の薄い餌場がある

問うもの:

- いつ離れるか
- どれくらい滞在するか
- 内部状態で切替が起きるか

Environment B: Two-Patch Tradeoff

- 高品質で危険なパッチ
- 低品質で安全なパッチ
- 枯渇速度が違う

問うもの:

- 状態依存で選択が変わるか
- 探索圧が立ち上がるか

Environment C: History-Dependent World

- 現在入力と同じ
- 直前履歴で有利行動が変わる

問うもの:

- 本当に記憶様状態を使っているか

この3つで十分です。

既存RL環境を使うなら

既存環境なら、完全には合いませんが、

- partially observable gridworld
- foraging environment
- multi-armed bandit with depletion and switching cost
- active inference 系の小環境

が近いです。

ただ、普通の bandit だと身体性が弱い。
普通の gridworld だと「ゴール到達」に寄りすぎる。
だからやはり自作がよいです。

一番大事なこと

この最小モデルの実証で問うべきは、

うまく解けたか

ではなく

内部状態とモードを入れないと出ない振る舞いが出たか

です。

なので、環境設計もそれに合わせて、

- 現在入力だけでは足りない
- 固定目標だけでは解けない
- 履歴と切替が必要

になるようにすべきです。

かなり圧縮すると、最初の環境はこれで十分です。

「**枯渇する餌パッチと危険刺激を持つ2D foraging 環境**」

これが、あなたの最小式を最も自然に試せます。

次に必要なら、

この環境を **状態変数・観測・行動・評価指標つきで、そのまま実装仕様** に落とします。



リポジトリつくって、そのURL渡したらCodexに渡して実装できますか？

おすすめの開発は？VSCodeにあなたが、Codex用のPoC計画を書いてもらうでもいいですが。

はい。かなりやりやすい進め方です。

リポジトリを作って、そのURLをCodex側で扱える形にしておけば、Codexはリポジトリのコードを読んで、修正し、PRを作る流れまで持っていけます。Codex cloudはGitHubアカウント接続でリポジトリを扱え、PR作成まで想定されています。GitHub上でも@codex reviewでレビューさせたり、GitHub ActionでCodexをCI/CDに組み込むこともできます。 [OpenAI Developers+2](#)

ただ、あなたのケースでは、最初から「Codexに丸投げ」より、**PoCの骨格を先に固定してからCodexに実装させる**ほうがいいです。

理由は単純で、このテーマは「最小知性」の定義がずれると、コードはすぐ増えるのに本質から外れやすいからです。

私のおすすめはこうです。

いちばんよい進め方

Phase 1: 人間が仕様を固定

まず 1 枚の設計書だけ作る。

入れるべきものはこれだけです。

- モデル
 - `h_t`: 遅い内部状態
 - `m_t`: 行動モード
- 環境
 - 2D の餌パッチ
 - 枯渴
 - 危険刺激
- 観測
 - 局所餌濃度
 - 危険
 - 自己状態の一部
- 行動
 - 前進
 - 旋回
 - 停留
- 評価
 - stay/leave
 - explore/exploit 切替
 - 履歴依存性
 - アブレーション

Phase 2: Codex に最小 PoC を作らせる

最初はこれだけ。

- シミュレータ
- 最小 2 変数モデル
- 可視化
- ログ出力
- 3 本の実験スクリプト
- README

Phase 3: Codex に比較実験をやらせる

比較対象は 3 つで十分です。

- 内部状態なし
- モードなし
- 2変数フルモデル

これで「本当に必要な要素」が見えます。

リポジトリ構成のおすすめ

かなり小さく始めるべきです。

```
minimal-agency/
  README.md
  docs/
    poc_plan.md
    model_spec.md
  src/
    env.py
    agent.py
    train.py
    eval.py
    viz.py
  configs/
    base.yaml
    ablation_no_h.yaml
    ablation_no_m.yaml
  experiments/
    run_baseline.sh
    run_ablation.sh
  tests/
    test_env.py
    test_agent.py
```

このくらいで十分です。

Codex に渡すときの依頼文の形

Codex は、雑な依頼より、**制約つきの実装依頼**のほうが強いです。

例えばこうです。

このリポジトリで、最小の生体的エージェンシー PoC を実装してください。

目的:

- 2変数モデル h , m を持つエージェントを作る
- 2D の枯渇する餌パッチ環境で動かす
- stay/leave と explore/exploit の切替を観察する
- ablation で h と m の必要性を検証する

制約:

- Python
- 依存は numpy, matplotlib, pyyaml 程度
- 強化学習ライブラリは使わない
- コードは小さく保つ
- README に実行方法を書く
- 可視化を必須にする

成功条件:

- 単一スクリプトで学習またはシミュレーションが走る
- エージェント軌跡が描画される
- フルモデルと ablation の差が見える

この粒度がよいです。

VS Code でやるなら

VS Code 自体は問題ありません。

ただ、Codex 連携の観点では、**GitHub で管理しやすい形にしておくことが重要です**。OpenAI の公式ドキュメントでも、Codex は GitHub 接続、GitHub 上のレビュー、GitHub Action での実行が整備されています。CLI もあり、macOS/Linux が主対応、Windows は実験的で WSL 推奨です。 [OpenAI Developers+3](#)

なのでおすすめは、

VS Code で仕様と初期ファイルを置く → GitHub に push → Codex に実装・修正・PR をさせるです。

どの開発スタイルがよいか

あなたのテーマだと、これが一番よいです。

- A. 先に PoC 計画を書く
- B. その計画を Codex に食わせる
- C. Codex には「まず最小版だけ」と指示する
- D. 追加はアブレーション単位で刻む

逆に悪い進め方は、

- いきなり大きい環境
- 学習器も環境も全部入り
- 生体らしさを最初から詰め込む

です。ほぼ確実にぶれます。

私の一番おすすめ

最初にあなたがやるべきなのは、リポジトリ作成そのものより、**docs/poc_plan.md を1本作る**ことです。

その中身はこれだけでいいです。

1. 目的
2. 最小モデル
3. 環境

4. 評価指標
5. アブレーション
6. 成功条件

これを置けば、Codex に渡す仕事はかなり明確になります。

必要なら次に、そのまま **repo** に置ける `poc_plan.md` と `README.md` のたたき台を書きます。



リポジトリ名はなにがいいでしょう？

まず、汎用すぎる名前は避けたほうがいいです。

`minimal-intelligence` や `bio-ai` は埋もれやすいし、あとで意味がずれます。

あなたの今の核は

- 最小
- 生体的
- エージェント / 整合性維持
- 状態依存
- 閉ループ

なので、そこが残る名前がよいです。

私なら候補はこのあたりです。

一番おすすめ

- `minimal-agency`
- `embodied-agency`
- `integrity-driven-agent`

今回の議論にかなり合う

- `state-dependent-agency`
- `closed-loop-agency`
- `coherence-driven-agent`
- `agency-from-state`

少し研究プロジェクトっぽい

- `proto-agency`
- `basal-agency`
- `minimal-organismic-agent`

- ecological-agency

線虫に寄せすぎず、本質寄り

- minimal-enactive-agent
- sensorimotor-agency
- integrity-loop
- adaptive-agency-core

私なら、今の段階ではこの3つに絞ります。

1. minimal-agency

一番無難で強いです。

短いし、今の目的とずれにくい。

あとで中身が広がっても耐える。

2. state-dependent-agency

今回の整理をかなり正確に表しています。

ただし少し長い。

3. integrity-driven-agent

「整合性駆動」を前面に出せる。

概念的にはかなりよいですが、英語としては少し硬い。

個人的には、**研究の入口としては minimal-agency が最強**だと思います。

理由は、今後

- 線虫風環境
- モード切替
- 内部状態
- 予測と記憶
- 階層化

のどれに広がっても、名前が邪魔しないからです。

逆に、今は避けたほうがいい名前もあります。

- worm-agent
線虫そのものが目的ではないので狭すぎる
- predictive-agent
予測だけが本質ではない
- homeostasis-agent
探索性が落ちる

- **transformer-agency**

実装方式に引っ張られすぎる

かなり率直に言うと、最初の repo 名は思想のラベルであって、完成品の説明である必要はないです。なので、迷うなら **minimal-agency** で始めるのがよいです。

必要なら次に、GitHub の description まで含めて **repo 名 + README 冒頭文** をセットで決めます。



minimal-enactive-agent にします。

description 準備してください。そのあとリポジトリつくって、Codex でそのあとの、plan を書いて、実装に進んでもらおうと思います。

いい名前です。**minimal-enactive-agent** なら、線虫専用でも予測専用でもなく、今の思想にかなり合っています。

GitHub の description 候補はこのあたりです。

第一候補

A minimal embodied agent for exploring state-dependent coherence-driven behavior in closed-loop environments.

少し平易

A minimal embodied agent for studying how behavior emerges from internal state in a closed sensorimotor loop.

研究色を強める

A proof-of-concept repository for minimal enactive agency: internal state, mode switching, and closed-loop behavior.

私なら、最初はこれを勧めます。

Description

A minimal embodied agent for studying how behavior emerges from internal state in a closed sensorimotor loop.

README の冒頭に置く短い説明も用意しておく と、Codex に渡しやすいです。

README 冒頭案

minimal-enactive-agent is a proof-of-concept project for building the smallest useful model of enactive agency.

The core idea is that behavior should emerge from a closed body-environment loop and state-dependent coherence dynamics, rather than from an externally fixed goal alone.

The first implementation focuses on a simple agent with a slow internal state, a behavior mode variable, and a minimal foraging environment with depletion and risk.

Codex 用に repo の最初の plan 名も揃えておく とよいです。

- **docs/poc_plan.md**

- docs/model_spec.md
- docs/experiment_plan.md

最初に Codex へ渡す一文も置いておきます。

Codex 向け冒頭指示

Implement the first proof of concept for a minimal enactive agent. The goal is not to maximize benchmark reward, but to demonstrate state-dependent behavior, mode switching, and closed-loop adaptation in a simple foraging environment.

必要なら次に、そのまま貼れる README.md 初版と docs/poc_plan.md 初版をまとめて書きます。



<https://github.com/demerzel09/minimal-enactive-agent.git>

作成しました。

Codexにこのまま移動できますか？

はい。

その GitHub リポジトリなら、**Codex cloud 側で GitHub を接続して、この repo を開いて進める**流れで行けます。Codex cloud は GitHub 接続でリポジトリを扱い、コードの読み書きや PR 作成までできます。GitHub 上で **@codex review** を使う連携もあります。 [OpenAI Developers+1](#)

ただし、ここから私が会話の中で**そのまま Codex 画面へ移動して実行することはできません**。あなた側で Codex を開いて、この repo を選び、最初のタスクを投げる必要があります。Codex は IDE、CLI、クラウドの各入口があり、VS Code 連携も公式に案内されています。 [OpenAI Developers+1](#)

進め方はこれが一番よいです。

1. Codex で **demerzel09/minimal-enactive-agent** を開く
2. 最初は実装ではなく、**docs/poc_plan.md** と **README.md** の初版作成だけ依頼する
3. その次に、最小環境と最小 2 変数エージェントの PoC 実装へ進める

最初に Codex に渡す指示は、このまま使えます。

Repository: demerzel09/minimal-enactive-agent

Create the initial project plan and scaffold for a proof of concept of a minimal enactive agent.

Goal:

Build a minimal embodied agent whose behavior emerges from:

- a closed body-environment loop
- state-dependent coherence dynamics

Do not optimize for generic benchmark performance.

Optimize for clarity, minimality, and experimental interpretability.

First task:

1. Create docs/poc_plan.md
2. Create docs/model_spec.md
3. Create a concise README.md
4. Propose a minimal repo structure for the first PoC

PoC requirements:

- Agent has two core state variables:
 - `h_t`: slow internal state
 - `m_t`: behavior mode
- Environment is a simple 2D foraging world with:
 - depleting food patch
 - sparse risk signal
- Evaluation should focus on:
 - stay/leave behavior
 - explore/exploit switching
 - history dependence
 - ablation of `h` and `m`

Constraints:

- Keep dependencies minimal
- Prefer Python + numpy + matplotlib
- Do not introduce large RL frameworks yet
- Keep the first implementation small and easy to inspect

Deliverables for this task:

- docs/poc_plan.md
- docs/model_spec.md
- updated README.md
- proposed src/ and configs/ layout

その次の実装フェーズ用には、続けてこう投げるとぶれにくいです。

Implement the first PoC described in docs/poc_plan.md.

Requirements:

- simple 2D environment
- minimal 2-variable agent
- trajectory visualization
- experiment scripts for full model vs ablations
- no heavy frameworks
- code should be short, readable, and easy to modify

先に repo の description も入れるなら、これを推します。

A minimal embodied agent for studying how behavior emerges from internal state in a closed sensorimotor loop.

必要なら次に、Codex に最初にかける README.md と docs/poc_plan.md の完成版をこちらで先に作れます。



これまでの議論のエッセンスも経緯として、どこかドキュメントに残したいですね。mdでまとめられますか？

はい。リポジトリにそのまま置ける形で、まずは docs/discussion_summary.md の初版として使えるものをまとめます。

Markdown

Discussion Summary: Minimal Enactive Agent

1. Background

This project did not start from a standard AI question such as “how to optimize behavior for a fixed goal.”

It started from a different question:

> When an external goal is not explicitly given, what does a system internally optimize in order to select the next action candidate?

This was motivated by a very ordinary human experience:

> "What should I do today?"

This is not just goal-directed planning.
It is closer to the emergence of a goal itself.

The hypothesis developed through discussion is that this kind of behavior requires more than explicit reasoning.
It requires a mechanism that integrates:

- current bodily and environmental conditions
- internal state
- past history
- tendencies and habits
- implicit valuation
- action readiness
- context-dependent behavioral switching

This led to the idea that the problem is not best framed as generic intelligence, but as a minimal form of **biological agency**.

2. Shift in framing

At first, the discussion used terms such as:

- intrinsic value signal
- minimal intelligence
- prediction
- memory
- mode switching
- internal state

However, these concepts overlapped and were not at the same level of abstraction.

The discussion gradually converged on a cleaner formulation.

Final high-level framing

Minimal biological agency can be understood through two core concepts:

1. **Body-environment closed loop**
2. **State-dependent coherence drive**

3. Core concepts

3.1 Body-environment closed loop

The system is not treated as a disembodied optimizer.
It exists in a loop:

- it senses
- it acts
- the environment changes
- the changed environment alters future input

Agency is therefore not an internal computation alone.
It is a dynamical relation between body and environment.

3.2 State-dependent coherence drive

When an external goal is not fixed, the system still tends to generate a "next action."
This is not modeled as generic reward maximization.

Instead, the system is assumed to shift toward action tendencies that are more coherent with its current internal state and ongoing body-environment relation.

"Coherence" here does **not** mean logical consistency.
It means something closer to:

- not collapsing
- not getting stuck

- maintaining viability
- preserving the possibility of continued interaction
- balancing local stability and continued exploration

A more operational interpretation is:

> coherence = maintaining viability while preserving exploratory continuity

4. Reinterpretation of value judgment

Under this framework, value judgment is not treated as an independent primitive.

Instead:

> Value judgment is the local ranking of action candidates produced by state-dependent coherence drive.

In other words:

- the closed loop is the arena
- coherence drive is the principle
- value judgment is the local expression of that principle during action selection

This avoids reducing the problem too early to a single scalar reward signal.

5. Why prediction and memory appeared in the discussion

The project then turned to the question:

> Are prediction and memory necessary for biological agency to become more sophisticated?

The discussion reached a layered answer.

5.1 At minimal levels

In worms, plants, or even simpler systems, what matters first may not be explicit prediction in a human-like sense.

What matters more fundamentally is:

- history-dependent state update
- persistence
- adaptive switching
- environment-coupled regulation

In such systems, prediction and memory may exist only in weak or implicit forms.

5.2 At human levels

For human-like cognition, prediction and memory appear to be indispensable.

Not necessarily in the form of explicit symbolic modules, but at least as:

- persistence of past influence on present state
- anticipatory adjustment of action
- reuse of past structure for future construction
- comparison of possible future states

Thus:

- **minimal agency** may exist without rich human-like prediction/memory
- **human-like cognition** almost certainly cannot

6. Continuous but non-identical relation across organisms

The discussion considered worms, slime molds, plants, and humans.

The resulting view is:

- there is likely a **continuity of principle**
- but not an identity of mechanism

For example:

- worms likely have history-dependent state transitions and weak anticipatory structure
- humans have layered memory, distributed prediction, reconstruction, and counterfactual comparison

So the project should not try to copy worms as worms.
It should try to identify the **essential mechanism** that remains meaningful across scales.

7. Minimal formal model

The earlier discussion gradually compressed the architecture to a minimal two-state form.

Practical minimal form

```
\[
h_{t+1} = f(h_t, i_t, m_t)
\]
```

```
\[
m_{t+1} = g(m_t, h_t, i_t)
\]
```

```
\[
a_t = \phi(m_t)
\]
```

Where:

- (h_t) : slow internal state
- (m_t) : behavior mode
- (i_t) : body-environment input
- (a_t) : action

Interpretation

- h_t holds slow internal bias, persistence, or history dependence
- m_t represents current behavior mode or action tendency
- the update rules implement state-dependent coherence drive
- action is generated from the current mode

This form was preferred because it is:

- compact
- inspectable
- suitable for minimal implementation
- expressive enough to produce state-dependent switching

8. Why this is not yet "human intelligence"

A key point from the discussion:

This minimal module may be a good **proto-agent** or **proto-cognitive unit**,
but it is probably not sufficient for human-like intelligence.

Human cognition likely requires additional structure such as:

- multi-timescale hierarchy
- shared latent state across subsystems
- re-usable and reconstructive memory
- explicit or implicit comparison of alternative futures
- counterfactual evaluation
- long-range integration across modalities and contexts

So:

> the minimal model is a candidate ancestor-form, not a full account of human intelligence

9. Relation to current ANN / Transformer models

The discussion also asked whether modern Transformers already possess this structure.

The conclusion was:

- Transformers do implement a form of context-dependent state update
- but they do **not** intrinsically implement biological agency in the above sense

Main reasons:

- no intrinsic body-environment closed loop by default
- no explicit slow internal viability-oriented state
- no clear distinction between local mismatch and global arbitration
- no endogenous coherence drive comparable to biological regulation

Therefore, Transformers may contain a **partial formal resemblance**, but they do not yet realize the intended structure.

10. Implementation direction

The project should begin with a small proof of concept rather than a large benchmark.

Recommended first target

A minimal embodied agent with:

- a slow internal state `h`
- a behavior mode variable `m`
- a simple closed-loop environment

Recommended environment

A simple 2D foraging environment with:

- depleting food patch
- sparse risk signal
- local sensing only
- no fixed external goal state

Evaluation should focus on:

- stay / leave behavior
- explore / exploit switching
- history dependence
- effect of ablations
- emergence of mode persistence and switching

The point is not benchmark score.

The point is to show that the added structure produces behaviors that cannot be reduced to immediate stimulus-response.

11. Working research stance

The project should avoid getting trapped in broad philosophical disputes such as:

- what exactly counts as intelligence
- whether plants are intelligent
- whether slime molds "really think"

Instead, a more useful stance is:

> build the minimal mechanism of biological agency first, then ask what level of cognition it supports

This suggests using working terms such as:

- minimal enactive agency
- biological agency
- coherence-maintaining control

- state-dependent closed-loop adaptation

rather than making strong claims too early.

12. Current distilled thesis

At the current stage, the most distilled statement is:

> Minimal biological agency is the capacity to generate the next action through state-dependent coherence drive within a body-environment closed loop.

And the practical implementation hypothesis is:

> A minimal agent may only need a slow internal state, a behavior mode variable, and a closed-loop environment to exhibit the earliest meaningful form of state-dependent agency.

13. Open questions

The discussion left several important open questions.

Conceptual

- Is "coherence" best defined as viability maintenance, exploratory continuity, or something more precise?
- At what point does history-dependent state update become genuine prediction?
- At what point does minimal agency become cognition?

Computational

- Is the two-variable model sufficient to produce robust mode switching?
- Is explicit prediction necessary in the first implementation?
- Can local mismatch and global arbitration be added later without losing minimality?

Comparative

- Which aspects of worm behavior are best seen as state transition rather than prediction?
- Which features of human cognition require qualitative additions beyond the minimal module?

14. Immediate next step

The immediate next step is not to solve cognition in general.

It is to implement a small closed-loop proof of concept that tests this claim:

> A minimal internal state plus behavior mode dynamics can generate coherent, history-dependent action selection without relying on an externally fixed goal.

That proof of concept is the purpose of this repository.

このあと必要なら、続けて以下のどちらかをそのまま作れます。

- **README.md** 向けに短く圧縮した版
- Codex に読ませやすいように、実装観点だけ残した **docs/poc_plan.md** 版



はい。その2つもつくって

以下をそのまま使えます。

README.md

Markdown

minimal-enactive-agent

A minimal embodied agent for studying how behavior emerges from internal state in a closed sensorimotor loop.

Overview

This repository explores a simple question:

> How can an agent generate the next action when no explicit external goal is fixed?

Instead of treating behavior as pure reward maximization, this project starts from two working ideas:

1. **Body-environment closed loop**
The agent senses, acts, changes the environment, and is changed by it in return.
2. **State-dependent coherence drive**
The agent tends to generate actions that are more coherent with its current internal state and ongoing interaction with the environment.

The goal is not to reproduce full intelligence.

The goal is to build the smallest useful computational model of **minimal enactive agency**.

Core hypothesis

A meaningful proto-agent may emerge from only:

- a **slow internal state**
- a **behavior mode variable**
- a **closed-loop environment**

This is meant to test whether simple state-dependent dynamics can generate:

- stay / leave behavior
- explore / exploit switching
- history dependence
- mode persistence and switching

without relying on a fixed external objective.

Minimal model

The practical minimal form currently considered is:

$$\begin{aligned} \backslash[\\ h_{t+1} = f(h_t, i_t, m_t) \\ \backslash] \end{aligned}$$

$$\begin{aligned} \backslash[\\ m_{t+1} = g(m_t, h_t, i_t) \\ \backslash] \end{aligned}$$

$$\begin{aligned} \backslash[\\ a_t = \phi(m_t) \\ \backslash] \end{aligned}$$

Where:

- $\backslash(h_t \backslash)$: slow internal state
- $\backslash(m_t \backslash)$: behavior mode
- $\backslash(i_t \backslash)$: body-environment input
- $\backslash(a_t \backslash)$: action

This model is intentionally small, interpretable, and easy to ablate.

First proof of concept

The first PoC should implement:

- a minimal 2D foraging environment
- a depleting food patch
- a sparse risk signal

- local sensing only
- an agent with internal state `h` and mode variable `m`

Evaluation focus

This repository does **not** optimize for generic benchmark performance.

The main questions are:

- Does internal state matter?
- Does mode dynamics matter?
- Can history-dependent behavior emerge?
- Can the agent show nontrivial stay/leave and explore/exploit patterns?

Repository direction

The project is intended to proceed in small steps:

1. define the minimal model clearly
2. build a minimal environment
3. implement the smallest working agent
4. run ablations
5. expand only when necessary

Working terminology

To avoid unnecessary philosophical inflation, this project uses terms such as:

- minimal enactive agency
- biological agency
- state-dependent coherence
- closed-loop adaptation

rather than claiming full intelligence from the outset.

Documents

- `docs/discussion\_summary.md` – background and conceptual path
- `docs/poc\_plan.md` – implementation-oriented proof-of-concept plan
- `docs/model\_spec.md` – model details and assumptions
- `docs/experiment\_plan.md` – evaluation plan

Status

Initial conceptual phase.

The next step is to implement the smallest inspectable proof of concept.

docs/poc_plan.md

Markdown

PoC Plan: Minimal Enactive Agent

1. Purpose

Build a first proof of concept for a minimal embodied agent whose behavior emerges from:

- a **body-environment closed loop**
- **state-dependent coherence dynamics**

The point is not to maximize benchmark reward.

The point is to test whether a very small internal architecture can generate meaningful, history-dependent action selection.

2. Main research question

> Can a minimal agent with a slow internal state and a behavior mode variable produce coherent closed-loop behavior without relying on a fixed external goal?

More concretely:

- Can the agent switch between staying and leaving?
- Can it shift between exploration and exploitation?
- Can current behavior depend on past interaction history?

- Do these behaviors disappear under ablation?

3. Minimal agent design

State variables

The first PoC uses two core variables:

- ``h_t``: slow internal state
- ``m_t``: behavior mode variable

Interpretation

``h_t`` – slow internal state

Represents slowly varying internal bias, such as:

- persistence of recent experience
- depletion pressure
- exploratory pressure
- internal tendency shaped by history

This is not yet meant to model a specific biological variable.
It is a compact state carrier.

``m_t`` – behavior mode

Represents the current behavioral regime, for example:

- exploit / stay
- explore / leave
- avoid / escape

The first implementation may use either:

- a continuous vector with soft competition, or
- a small discrete-like softmax mode state

Action output

Actions may be minimal, for example:

- move forward
- turn left
- turn right
- stay / pause

The exact set should remain small.

4. Minimal model form

A practical minimal update structure:

```
\[
h_{t+1} = f(h_t, i_t, m_t)
\]
```

```
\[
m_{t+1} = g(m_t, h_t, i_t)
\]
```

```
\[
a_t = \phi(m_t)
\]
```

Where:

- ``i_t`` is body-environment input
- ``a_t`` is action

Recommended first implementation form

Use a small recurrent rate-based model.

Example structure:

- ``h`` updated with a slow leak

- `m` updated with a faster leak
- `m` converted to action preference
- no explicit prediction module at first
- no heavy learning framework at first

5. Environment

5.1 First environment: depleting food patch

A 2D world containing:

- one main food patch
- food depletion when the agent remains nearby
- sparse risk signal
- optional weak background food elsewhere

Why this environment

It directly probes the behaviors of interest:

- when to stay
- when to leave
- when to switch mode
- whether history matters

It avoids overfitting the project to standard goal-reaching tasks.

5.2 Observation

The agent should receive only local or near-local information:

- local food intensity
- local risk intensity
- possibly local gradient estimate
- optional proprioceptive or previous-action signal

The agent should not receive the full global map.

5.3 Environment update

The environment should update after each action:

- food is partially consumed or depleted
- the agent position changes
- risk may fluctuate or remain sparse
- the resulting observation becomes the next input

This preserves the body-environment closed loop.

6. Implementation constraints

The first PoC should be intentionally small.

Preferred stack

- Python
- numpy
- matplotlib
- pyyaml if config files are used

Avoid for now

- large RL frameworks
- deep learning libraries unless clearly needed
- complicated training pipelines
- benchmark-heavy engineering

The first version should be inspectable and easy to modify.

7. Evaluation

7.1 Primary evaluation questions

We are not mainly asking "How high is reward?"

We are asking:

- Does internal state make a behavioral difference?
- Does mode structure make a behavioral difference?
- Does the system show nontrivial persistence and switching?
- Does it behave differently under different histories despite similar immediate input?

7.2 Suggested metrics

Behavioral

- average patch residence time
- leave probability as a function of local depletion
- frequency of mode switching
- duration of mode persistence
- path tortuosity / exploration spread
- time spent near risk vs food

History dependence

- whether current action distribution depends on recent interaction history
- whether leave behavior changes after prolonged low yield
- whether behavior differs under matched current observations but different recent pasts

Robustness

- behavior under input noise
- sensitivity to parameter changes

7.3 Ablation tests

These are essential.

Full model

- slow internal state `h`
- mode variable `m`

Ablation A: no internal state

- remove `h`
- keep only immediate input and mode dynamics

Question:

- does behavior collapse toward stimulus-response?

Ablation B: no mode variable

- remove `m`
- act directly from internal state and input

Question:

- does persistence / switching degrade?

Ablation C: reduced recurrence

- weaken or remove self-recurrence

Question:

- does history dependence disappear?

8. Expected outcome of first PoC

Success does **not** require biological realism or high task performance.

A successful first PoC should show at least some of the following:

- state-dependent stay/leave behavior
- nontrivial explore/exploit switching
- mode persistence over time
- history-dependent behavior not reducible to immediate observation
- clear degradation under ablations

9. Deliverables for the first coding phase

Core code

- `src/env.py` – minimal 2D foraging environment
- `src/agent.py` – minimal 2-variable agent
- `src/run\_simulation.py` – run one simulation
- `src/eval.py` – compute simple metrics
- `src/viz.py` – trajectory and state visualization

Configuration

```
- `configs/base.yaml`
- `configs/ablation_no_h.yaml`
- `configs/ablation_no_m.yaml`
```

Documentation

```
- update `README.md`
- add model notes to `docs/model_spec.md`
- add experiment notes to `docs/experiment_plan.md`
```

10. Recommended development order

Phase 1

Implement the environment only.

Phase 2

Implement a fixed hand-tuned agent with the two-state update.

Phase 3

Visualize trajectories, modes, and internal state.

Phase 4

Run ablations.

Phase 5

Only after that, consider parameter search or learning.

11. Non-goals for the first PoC

The first PoC should **not** attempt to do all of the following:

- full biological realism
- full worm behavior reproduction
- human-like cognition
- explicit world modeling
- large-scale memory systems
- full predictive coding implementation
- complex multi-agent interaction

These may become later directions, but they are not required for the first proof of concept.

12. Working stance

The project is trying to identify an essential mechanism, not to imitate a specific organism directly.

The current working stance is:

> If a tiny recurrent agent with a slow internal state and mode dynamics can already produce coherent closed-loop behavior, then it may capture a meaningful proto-form of enactive agency.

This is the claim the first PoC should test.

必要なら次に、**docs/model_spec.md** と **docs/experiment_plan.md** もこの流れで続けて作れます。



はいお願いします。

以下をそのまま使えます。

docs/model_spec.md

Markdown

Model Specification: Minimal Enactive Agent

1. Purpose

This document specifies the first computational model used in `minimal-enactive-agent`.

The model is intentionally minimal.

Its purpose is not to reproduce a specific organism, nor to maximize benchmark reward.

Its purpose is to test whether a very small recurrent agent can generate meaningful state-dependent behavior in a closed body-environment loop.

2. Design principles

The model is built from two working principles:

1. ****Body-environment closed loop****

The agent senses, acts, changes the environment, and then receives changed input.

2. ****State-dependent coherence drive****

The agent's next action tendency depends on its current internal state and ongoing interaction history, rather than only on immediate input or an externally fixed goal.

3. Minimal state structure

The agent uses two core state variables:

- `h\_t`: slow internal state
- `m\_t`: behavior mode state

3.1 Slow internal state `h\_t`

`h\_t` is the minimal carrier of persistence and history dependence.

It is meant to capture effects such as:

- recent success or failure
- depletion pressure
- exploratory pressure
- internal drift caused by past interaction
- slowly changing behavioral bias

`h\_t` should evolve more slowly than `m\_t`.

This variable is deliberately abstract.

In the first PoC it is not assigned a single biological interpretation such as hunger or arousal. It is a compact dynamical state.

3.2 Behavior mode `m\_t`

`m\_t` represents the agent's current behavioral regime.

In the first implementation, it may correspond loosely to a small set of modes such as:

- exploit / stay
- explore / leave
- avoid / escape

`m\_t` should evolve faster than `h\_t`, but still retain persistence.

The project does not assume that mode must be a hard discrete state.

A soft mode representation is acceptable and may be preferable for implementation.

4. Input and action

4.1 Input `i\_t`

`i\_t` denotes body-environment input available at time `t`.

Typical components may include:

- local food intensity
- local risk intensity
- local food gradient estimate
- previous action
- optional proprioceptive signal

The agent should not observe the full environment state.

4.2 Action a_t

a_t denotes the action produced at time t .

The first implementation should use a small action set, such as:

- move forward
- turn left
- turn right
- pause / stay

If continuous movement is easier, action can be represented as:

- heading change
- movement magnitude

The action space should remain simple.

5. Update equations

The practical minimal model is:

$$h_{t+1} = f(h_t, i_t, m_t)$$

$$m_{t+1} = g(m_t, h_t, i_t)$$

$$a_t = \phi(m_t)$$

This structure should be interpreted as follows:

- h accumulates slow internal history
- m converts current state plus input into mode dynamics
- a is generated from mode state

6. Recommended implementation form

The first implementation should use a small recurrent rate-based model.

A recommended form is:

$$h_{t+1} = (1 - \alpha_h) h_t + \alpha_h \tanh(W_{hh} h_t + W_{hm} m_t + W_{hi} i_t + b_h)$$

$$u_{t+1} = (1 - \alpha_m) u_t + \alpha_m \tanh(W_{uh} h_t + W_{uu} u_t + W_{ui} i_t + b_u)$$

$$m_t = \text{softmax}(u_t)$$

$$a_t = W_a m_t + b_a$$

Where:

- h_t is the slow internal state vector
- u_t is the pre-softmax mode activation
- m_t is the behavior mode distribution
- a_t is the action output
- $\alpha_h < \alpha_m$ so that h is slower than mode activation

6.1 Why this form

This form is preferred because it gives:

- recurrence
- persistence
- mode competition
- internal-state dependence
- simple inspectable dynamics

without requiring large frameworks or biologically detailed neuron models.

7. Minimal dimensionality

The first PoC should remain small.

Suggested starting dimensions:

- h : 1 to 4 dimensions
- m : 2 to 4 modes
- i : 2 to 5 dimensions depending on environment design

A good starting point might be:

- h = 2 dimensions
- m = 3 modes
- i = 3 inputs

Example interpretation:

- $h[0]$ = depletion / dissatisfaction tendency
- $h[1]$ = exploratory drift
- m = [exploit, explore, avoid]

These interpretations are optional.

The implementation should remain flexible.

8. Time-scale assumption

A central hypothesis of the model is that behavior emerges from interaction between two time scales:

- **slow** internal drift and persistence
- **faster** mode-level switching

This distinction is important.

If everything updates at the same time scale, the system may collapse toward immediate reaction. If internal state is too slow or too disconnected, it may become behaviorally irrelevant.

9. What counts as memory here

In this model, memory is not introduced as a separate symbolic memory module.

Instead, memory is implemented minimally as:

- persistence of h_t
- recurrence in m_t
- dependence of future state on past interaction

This is intentional.

The first PoC aims to test whether minimal history dependence is enough to produce interesting behavior.

10. What counts as prediction here

The first PoC does not require an explicit predictive model.

Prediction, if present at all in the first stage, is only implicit in the recurrence and dynamics. The model may later be extended with:

```
\[
\hat{i}_{t+1} = P(h_t, m_t)
\]
```

```
\[
e_t = i_t - \hat{i}_t
\]
```

but this is not required initially.

The first model should stay simpler unless explicit prediction is necessary to reproduce the target behavior.

11. Learning vs hand-tuned dynamics

The first version does not need full learning.

Recommended order:

1. hand-tune weights and parameters
2. verify that interesting behavior is possible
3. add simple parameter search if needed
4. only later consider trainable optimization

This reduces confusion between:

- architecture effects
- training effects

The first goal is architectural validity.

12. Required qualitative properties

A successful model should exhibit at least some of the following:

- persistence in behavioral mode
- switching between modes under changing conditions
- sensitivity to recent history
- nontrivial stay/leave behavior
- nontrivial explore/exploit behavior
- degradation under ablations

13. Ablation targets

The model is only meaningful if its key parts can be tested by removal.

Ablation A: remove slow internal state

- set `h_t = 0` or bypass `h`

Expected question:

- does the agent become overly reactive?

Ablation B: remove mode dynamics

- bypass `m`
- map input directly to action

Expected question:

- does persistence disappear?

Ablation C: weaken recurrence

- reduce self-connections or leak persistence

Expected question:

- does history dependence degrade?

14. Non-claims

This model does not yet claim to capture:

- full biological realism
- worm-specific neural circuitry
- explicit cognitive planning
- episodic memory
- human-like prediction
- symbolic reasoning

It is a proto-agent model, not a full cognitive architecture.

15. Current working hypothesis

The working hypothesis of the model is:

> A small recurrent system with a slow internal state and a behavior mode variable may be sufficient to produce the earliest useful form of state-dependent enactive agency in a closed-loop environment.

This hypothesis is what the first PoC is meant to test.

docs/experiment_plan.md

Markdown

Experiment Plan: Minimal Enactive Agent

1. Purpose

This document defines the first experiments for `minimal-enactive-agent`.

The aim is not to optimize benchmark performance.

The aim is to determine whether the minimal architecture produces meaningful closed-loop, state-dependent behavior that cannot be reduced to immediate stimulus-response.

2. Main experimental question

> Does a minimal agent with a slow internal state and a behavior mode variable produce behavior that is persistent, history-dependent, and adaptively switchable in a simple closed-loop environment?

3. First experimental environment

3.1 Environment A: Depleting Food Patch

The first environment should be a small 2D world containing:

- one main food patch
- local food sensing
- depletion when the agent remains in or near the patch
- sparse risk signal
- weak or absent explicit target location

Rationale

This environment is chosen because it naturally probes:

- stay / leave behavior
- persistence
- history dependence
- exploration after local depletion
- interaction between exploitation and avoidance

It is better aligned with the project than fixed-goal navigation tasks.

3.2 Suggested environment mechanics

World

- 2D continuous plane or small discrete grid
- bounded domain

Food

- one region with initially high food intensity
- food depletes with local occupation or consumption
- optional slow regeneration

Risk

- one sparse risk source or stochastic risk field
- risk should not dominate the task
- risk exists to test switching and avoidance

Observation

- local food level
- local risk level
- local gradient estimate if desired
- optional previous action or heading

Action

- forward move
- turn left
- turn right
- pause / stay

4. Experimental conditions

4.1 Full model

This is the reference condition.

Agent includes:

- slow internal state `h`
- mode dynamics `m`
- recurrence
- closed-loop interaction

This condition tests whether the architecture can produce the target behaviors.

4.2 Ablation A: no internal state

Remove or freeze `h`.

Possible implementation:

- set `h\_t = 0`
- remove recurrence into `h`
- bypass `h` in downstream dynamics

Question:

- does the agent lose history dependence?

Expected failure mode:

- behavior becomes overly tied to immediate observation

4.3 Ablation B: no mode state

Remove or bypass `m`.

Possible implementation:

- map directly from input and optional `h` to action
- remove mode persistence

Question:

- does the agent lose stable action regimes?

Expected failure mode:

- behavior becomes noisy or shallowly reactive
- stay / leave switching becomes weak or erratic

4.4 Ablation C: reduced recurrence

Weaken self-recurrence or leak persistence.

Question:

- does mode persistence or internal continuity disappear?

Expected failure mode:

- behavior becomes unstable or memoryless

5. Core behavioral tests

5.1 Stay / leave test

Measure whether the agent remains in the food patch early, but leaves when local yield drops or risk rises.

Questions:

- does residence time depend on recent interaction history?
- does the leave probability increase after depletion?
- is leave timing more structured in the full model than in ablations?

5.2 Explore / exploit switching test

Measure whether the agent alternates between:

- local exploitation
- broader exploration

Questions:

- does switching emerge without a fixed external goal?
- do mode dynamics show persistence rather than random oscillation?
- do ablations reduce this structure?

5.3 History dependence test

Construct episodes in which immediate observation is similar, but recent history differs.

Example:

- same current food level
- different recent depletion trajectory

Questions:

- does action choice differ across histories?
- can the full model distinguish states that immediate input alone cannot?

5.4 Risk-sensitive switching test

Use sparse risk input to test whether the agent changes mode when exploitation becomes unsafe.

Questions:

- does the agent avoid pure greed?
- does risk influence mode switching, not just immediate turning?

6. Metrics

6.1 Basic behavioral metrics

- average residence time in food patch
- leave probability over time
- average food collected
- average risk exposure
- number of mode switches
- average mode duration
- path length
- exploration radius
- occupancy heatmap

These metrics are useful but not sufficient on their own.

6.2 History-sensitive metrics

- divergence in action distribution under matched current input but different recent history
- dependence of leave probability on recent yield trend
- dependence of mode occupancy on recent local depletion

These metrics are especially important because they test whether internal state matters.

6.3 Dynamics metrics

- temporal autocorrelation of mode variable
- temporal autocorrelation of action selection
- lagged coupling between internal state and mode transitions
- sensitivity of switching to slow-state drift

These help determine whether the model truly exhibits structured dynamics rather than random fluctuations.

7. Visualization plan

Visualization is essential for this project.

At minimum, each experiment should generate:

- trajectory plot
- food field snapshot or depletion map
- time series of `h`
- time series of `m`
- action sequence
- patch occupancy over time

Optional but useful:

- phase plot of `h` vs dominant mode
- comparison plots across ablations

- event markers for entering / leaving patch

8. Success criteria for the first PoC

The first PoC is successful if the full model shows at least some of the following:

- persistent behavioral modes
- nontrivial stay / leave behavior
- nontrivial explore / exploit switching
- dependence on recent interaction history
- clear qualitative degradation under ablation

A successful first PoC does ****not**** require:

- perfect optimization
- biological realism
- high reward score
- match to any one species

9. Failure cases and interpretation

9.1 If full model behaves like stimulus-response

Possible interpretation:

- internal state is too weak
- recurrence is too weak
- environment does not require history
- mode competition is poorly tuned

9.2 If full model is unstable or chaotic

Possible interpretation:

- recurrent gain too strong
- time-scale separation poorly chosen
- mode state too unconstrained

9.3 If ablations perform similarly to full model

Possible interpretation:

- environment is too easy
- architecture is unnecessarily complex
- internal state is not actually contributing
- mode variable is redundant in current setup

This would be an important result, not just a failure.

10. Recommended experimental order

Phase 1: Environment sanity check

- visualize food and risk fields
- verify depletion logic
- verify action and movement

Phase 2: Full model qualitative run

- run hand-tuned full model
- inspect trajectories and state traces

Phase 3: Ablation comparison

- no `h`
- no `m`
- reduced recurrence

Phase 4: Parameter sweep

- vary time constants
- vary recurrence strength
- vary mode competition strength

Phase 5: Stress tests

- add observation noise
- vary patch quality
- vary depletion rate
- vary risk strength

11. Comparison logic

The most important comparison is not raw score.
It is behavioral structure.

Preferred order of interpretation:

1. Does full model show the intended qualitative patterns?
2. Do ablations weaken those patterns?
3. Can differences be explained by internal state and mode dynamics?
4. Which architectural element contributes most?

12. Deliverables

Each experiment batch should produce:

- saved plots
- summary metrics
- configuration used
- short textual interpretation

Suggested output structure:

- `outputs/<experiment\_name>/plots/`
- `outputs/<experiment\_name>/metrics.json`
- `outputs/<experiment\_name>/config.yaml`
- `outputs/<experiment\_name>/notes.txt`

13. Future extensions after first PoC

Only after the first PoC is successful should the project consider:

- explicit prediction error
- learned parameters
- multiple food patches
- hierarchical modes
- multiple timescale internal states
- recurrent modular composition
- comparison to biological trajectory data

14. Current stance

The first experiments are intended to answer a narrow question:

> Is a tiny recurrent closed-loop agent with slow internal state and mode dynamics already enough to generate a meaningful proto-form of enactive agency?

This is the claim the first experiment suite should test.

次に進めるなら、Codex に投げる最初の実装指示文も、この4つのドキュメント前提で、ぶれにくい形に整えられます。



リポジトリ更新しました。確認して



<https://github.com/demerzel09/minimal-enactive-agent.git>

確認できました。README と docs は反映されています。 [GitHub](#)

見えている範囲では、少なくとも次が入っています。

- リポジトリ説明文
- README.md
- docs/
- docs/discussion_summary.md
- docs/poc_plan.md
- docs/model_spec.md
- docs/experiment_plan.md [GitHub](#)

README も意図どおり入っています。特に、

- body-environment closed loop
- state-dependent coherence drive
- 最小モデル h, m, i, a
- 最初の PoC の環境
- 評価観点
- docs への導線

が揃っています。 [GitHub](#)

1点だけ気になったのは、GitHub の表示上、数式がそのまま

- $[h_{t+1} = f(h_t, i_t, m_t)]$
- $[m_{t+1} = g(m_t, h_t, i_t)]$
- $[a_t = \phi(m_t)]$

のように見えていて、GitHub では綺麗な数式レンダリングにはなっていません。内容は通じますが、README では数式をコードブロックかプレーンテキストにしたほうが見やすいです。 [GitHub](#)

次は、Codex に渡す最初のタスクとしてはこの順がよいです。

1. src/, configs/, outputs/ の初期構成を作る
2. env.py, agent.py, run_simulation.py, viz.py の最小版を作る
3. full model と ablation 2種を走らせる
4. README に実行方法を追記する

Codex に投げる最初の指示文も、今の repo 状態に合わせてすぐ使える形に整えられます。



GitHub では綺麗な数式レンダリングにはなっていません これをキレイな式にするには？

GitHub では、`\[ ... \]` より `$$ ... $$` のほうが安定です。
README なら、まずこれに直すのがよいです。

今の

Markdown

```
[ h_{t+1} = f(h_t, i_t, m_t) ]
[ m_{t+1} = g(m_t, h_t, i_t) ]
[ a_t = \phi(m_t) ]
```

は、次のように書き換えます。

Markdown

```
$$
h_{t+1} = f(h_t, i_t, m_t)
$$

$$
m_{t+1} = g(m_t, h_t, i_t)
$$

$$
a_t = \phi(m_t)
$$
```

1つにまとめるなら、こうでもよいです。

Markdown

```
$$
\begin{aligned}
h_{t+1} &= f(h_t, i_t, m_t) \\
m_{t+1} &= g(m_t, h_t, i_t) \\
a_t &= \phi(m_t)
\end{aligned}
$$
```

本文中に入れる短い式なら、インラインで

Markdown

The agent uses h_t as a slow internal state and m_t as a behavior mode.
です。

注意点はこれです。

- `$$` の前後は空行を入れる
- README のプレビュー表示で確認する
- `\[ ... \]` や `[ ... ]` は避ける
- 数式だけを 1 行ずつ独立させると崩れにくい

GitHub 上でいちばん無難なのは、**インラインは $\dots$ 、独立式は $\dots$** です。

必要なら、README の数式部分だけ、そのまま貼れる形に整えて返します。



お願いします

もちろんです。README の数式部分だけ差し替えるなら、これをそのまま貼れば大丈夫です。

Markdown

Minimal model

The practical minimal form currently considered is:

```


$$\begin{aligned}
h_{t+1} &= f(h_t, i_t, m_t) \\
m_{t+1} &= g(m_t, h_t, i_t) \\
a_t &= \phi(m_t)
\end{aligned}$$


```

Where:

- h_t : slow internal state
- m_t : behavior mode
- i_t : body-environment input
- a_t : action

This model is intentionally small, interpretable, and easy to ablate.

GitHub でより崩れにくくするなら、箇条書き側も LaTeX ではなくプレーンにしてもいいです。こちらのほうが無難です。

Markdown

Minimal model

The practical minimal form currently considered is:

```


$$\begin{aligned}
h_{t+1} &= f(h_t, i_t, m_t) \\
m_{t+1} &= g(m_t, h_t, i_t) \\
a_t &= \phi(m_t)
\end{aligned}$$


```

Where:

- h_t : slow internal state
- m_t : behavior mode
- i_t : body-environment input
- a_t : action

This model is intentionally small, interpretable, and easy to ablate.

この方が GitHub 上では安定です。

必要なら、README 全体を GitHub 向けに少し整形した版も作れます。



discussion_summary.md の
7. Minimal formal mode
も書き換えたい。

はい。discussion_summary.md の 7. Minimal formal model は、今の整理に合わせてもっときれいにできます。

前よりも、

- 2変数版を中心にする
- 「最小知性」より「最小エージェンシー」に寄せる
- 方程式そのものより、その意味を短く明確にする

ほうがよいです。

差し替え案はこれです。

Markdown

7. Minimal formal model

The discussion gradually converged on a compact practical form for the first implementation.

Practical minimal form

```


$$\begin{aligned}
 h_{t+1} &= f(h_t, i_t, m_t) \\
 m_{t+1} &= g(m_t, h_t, i_t) \\
 a_t &= \phi(m_t)
 \end{aligned}$$


```

Where:

- h_t : slow internal state
- m_t : behavior mode
- i_t : body-environment input
- a_t : action

Interpretation

This form is meant to be the smallest practical expression of the project's current hypothesis.

- h_t carries slow persistence, internal bias, and history dependence
- m_t represents the current behavioral regime or action tendency
- i_t connects the agent to the body-environment loop
- a_t is generated from the current mode state

The update functions f and g are the computational locus of what we have called **state-dependent coherence drive**.

In this framing:

- memory is not a separate symbolic module
- prediction is not necessarily explicit
- value judgment is not a standalone scalar

Instead, persistence, switching, and action selection are all treated as consequences of recurrent state dynamics within a closed loop.

Why this form was preferred

This formulation was preferred because it is:

- compact
- interpretable
- easy to implement
- easy to ablate
- flexible enough to support later extensions

It is not intended as a full account of intelligence.

It is intended as the smallest useful formal core for testing **minimal enactive agency**.

Compressed view

At the most compressed level, the model says:

```
> an agent maintains an internal state, updates it through closed-loop interaction, and generates action through mode dynamics shaped by that state
```

This is the current minimal formal hypothesis of the project.

さらに少し強めに、今の思想へ寄せるなら、末尾をこう変えてもいいです。

Markdown

Compressed view

At the most compressed level, the model says:

```
> minimal enactive agency arises when a system maintains a slow internal state, expresses behavior through mode dynamics, and updates both through closed-loop interaction with the environment
```

This is the current minimal formal hypothesis of the project.

個人的には後者のほうが、この repo の名前とも合っています。



2つとも書き換えました。確認して

まだ反映されていません。

README.md は依然として `\[ ... \]` のままです。 `$$ ... $$` への書き換えは見えていません

[README](#)

docs/discussion_summary.md の 7. Minimal formal model も、以前の短い版のままで、差し替え案の本文には

なっていません [discussion_summary](#)

つまり、可能性はだいたいこのどれかです。

- ローカルでは直したが push していない
- 別ブランチに入っている
- GitHub 上で保存前の状態
- main ではなく別ブランチに入っている

確認ポイントはこれです。

Bash

```
git status
git branch
git add .
git commit -m "Update README math and discussion summary"
git push origin main
```

別ブランチなら、今いるブランチ名を確認して、そのブランチを push する必要があります。

必要なら次に、**今の main にまだ入っていない差分だけ**を、コピペしやすい形で短くまとめます。

情報源



プッシュ忘れてました。もう一度確認して

確認できました。反映されています。

README の数式は `$$ ... $$` に変わっていて、独立式として入っています

[README](#)

`discussion_summary.md` の 7. Minimal formal model も、新しい長めの版に置き換わっています [discussion_summary](#)

見た感じ、今の状態で十分よいです。

細かく言うと、`discussion_summary.md` の 7 節の末尾だけ、**This is the current minimal formal hypothesis of the project.** の直後に `---` が改行なしで続いているので、そこだけ 1 行空けると見た目が整います

[discussion_summary](#)

つまりここだけ、

Markdown

This is the current minimal formal hypothesis of the project.

8. Why this is not yet “human intelligence”

にするとよいです。

この状態なら、次は Codex に

- repo 構成作成
- 最小 PoC 実装
- ablation 実験

を投げてよい段階です。

情報源



このリポジトリ上で動かすCodexに最初に投げる指示文を作成して

これを最初の指示文としてそのまま Codex に渡すのがよいです。

Repository: demerzel09/minimal-enactive-agent

Read the existing documents first and use them as the source of truth:

- README.md
- docs/discussion_summary.md
- docs/poc_plan.md
- docs/model_spec.md
- docs/experiment_plan.md

Your task is to implement the first proof of concept for this repository.

Goal:

Build a minimal embodied agent whose behavior emerges from:

1. a body-environment closed loop
2. state-dependent coherence dynamics

This project is not about maximizing generic benchmark reward.

It is about building the smallest inspectable computational model that can show:

- stay / leave behavior
- explore / exploit switching
- history dependence
- mode persistence and switching

Please implement the first PoC with the following constraints.

Core agent requirements:

- Use the minimal two-state structure described in the docs
- Include:
 - `h_t`: slow internal state
 - `m_t`: behavior mode
- Keep the implementation small, explicit, and easy to inspect
- Prefer a simple recurrent rate-based implementation over heavy ML frameworks
- No transformer, no large RL library, no unnecessary abstractions

Environment requirements:

- Implement a simple 2D foraging environment
- Include:
 - one depleting food patch
 - sparse risk signal
 - local sensing only
- The environment must be closed-loop:
 - the agent acts
 - the environment changes
 - the next observation depends on that interaction

Code constraints:

- Prefer Python
- Prefer only lightweight dependencies such as:
 - numpy
 - matplotlib
 - pyyaml
- Keep the first implementation minimal and readable
- Avoid overengineering
- Do not add training infrastructure unless clearly necessary
- Start with hand-tuned or fixed dynamics if possible

Required outputs:

1. Create a minimal project structure under:
 - `src/`
 - `configs/`
 - `outputs/` (if needed)
2. Implement at least:
 - `src/env.py`
 - `src/agent.py`
 - `src/run_simulation.py`
 - `src/eval.py`
 - `src/viz.py`
3. Add config files for:
 - full model
 - ablation without `h`
 - ablation without `m`
4. Update README.md with:
 - how to run the simulation
 - how to run the ablations
5. Keep everything aligned with the docs already in the repository

Evaluation requirements:

The implementation should support evaluation of:

- stay / leave behavior
- explore / exploit switching
- history dependence
- effect of ablations

Do not optimize only for reward.

Make it possible to observe qualitative behavior through:

- trajectory plots
- time series of h
- time series of m
- simple metrics

Ablations:

Implement at least these three conditions:

- full model
- no internal state h
- no mode variable m

Development order:

1. implement the environment
2. implement the full minimal agent
3. add visualization
4. add evaluation
5. add ablations

Important:

- Keep the code small and inspectable
- Favor conceptual clarity over sophistication
- Do not drift away from the repository's core concepts
- If a design choice is ambiguous, choose the simpler option and document it briefly

Deliverables for this first step:

- working minimal code
- runnable simulation
- basic visualizations
- ablation configs
- updated README

Before coding, briefly summarize your implementation plan based on the repo documents, then proceed.

少し強めに「最初は実装せず plan だけ書かせたい」版もあります。

Repository: `demerzel09/minimal-enactive-agent`

Read the existing documents first and use them as the source of truth:

- README.md
- docs/discussion_summary.md
- docs/poc_plan.md
- docs/model_spec.md
- docs/experiment_plan.md

First, do not implement the full system yet.

Task:

Create a short implementation plan and scaffold for the first PoC.

Please do the following:

1. summarize the intended architecture in your own words
2. propose the concrete file structure
3. explain the simplest agent/environment design that matches the docs
4. identify any ambiguities and resolve them conservatively
5. then implement the minimal first version

Constraints:

- keep dependencies minimal
- prefer numpy + matplotlib
- no heavy RL frameworks
- keep the code small and readable
- align strictly with the repository documents

私なら最初は前者を使います。
Codex が大きく作りすぎる傾向があるなら、後者から入るのが安全です。



Codexは、一晩勝手に動くエージェントモードはありますか？であれば、テストまでして、翌朝確認したいです。

この会話からは、**私がそのまま一晩バックグラウンドで Codex を走らせることはできません。**

ただ、質問の意図はたぶん「Codex 自体に、非対話で進めるエージェント的な実行モードがあるか」ですね。

そこは、**手元のこの環境では最新仕様をライブ確認できないので断定は避けますが、実務上は次の理解で進めるのが安全です。**

- **この会話の私**には、夜間に放置して翌朝結果を返すような非同期実行能力はない
- **Codex 側**は、GitHub 連携や PR ベースの作業には向いている
- ただし「一晩勝手に探索・実装・修正・再テストを繰り返す完全自律モード」があるかは、**Codex の現行UI/プラン依存**なので、Codex 画面か公式 docs で確認する必要がある

実際の運用としては、翌朝確認したいなら、いちばん堅いのはこれです。

1. Codex に最初の実装をさせる
2. リポジトリに `pytest` や実験スクリプトを入れる
3. GitHub Actions でテストと実験を自動実行する
4. Codex には PR まで作らせる
5. 朝に PR と CI 結果を見る

つまり、「**夜間に自動で回す主体**」は **Codex 単体より CI に寄せる**ほうが確実です。

Codex はコード生成と修正、CI は継続実行と検証、という分担です。

あなたの repo なら、最初に入れるべき自動化はこの程度で十分です。

- `pytest` の最小テスト
- `python src/run_simulation.py --config configs/base.yaml`
- `python src/eval.py ...`
- 生成画像やメトリクスを artifact として保存

これなら、Codex が実装したあと、夜間に GitHub Actions で回して、翌朝に

- テストが通ったか
- シミュレーションが動いたか
- ablation の差が出たか

を見られます。

必要なら次に、この repo 用の GitHub Actions 設定と、Codex に渡す「CI 前提」の実装指示文をそのまま作ります。



Codex CLI

Codex CLI は、ターミナルから Codex に実装作業を投げるための CLI です。
あなたの使い方なら、かなり相性がいいです。

向いているのはこういう流れです。

- ローカルで repo を開く
- README.md と docs/ を読ませる
- 「この方針で最小 PoC を実装して」と指示する
- 変更差分を確認する
- 必要ならテストや実行もさせる

つまり、今回の minimal-enactive-agent みたいに、方針は人間が握って、実装と反復を Codex CLI にやらせるのに向いています。

ただし、一つ切り分けが必要です。
あなたがさっき気にしていた

一晩勝手に動くエージェントモード

については、CLI 自体が完全自律で朝まで勝手に長時間探索し続けるかは、環境設定や実行方法に依存します。

少なくとも実務的には、次の形にすると近い運用になります。

- Codex CLI に実装指示を出す
- 変更を repo に反映させる
- テストは pytest や実験スクリプトで回す
- 長時間実行は tmux / screen / CI で回す
- 朝に差分とログを見る

つまり、CLI を頭脳、シェルと CI を持続実行装置として使う感じです。

あなたの repo なら、Codex CLI には最初にこういう指示が合います。

```
Read README.md and all files in docs/ first.
Then implement the first proof of concept described there.
```

```
Goal:
- minimal embodied agent
- slow internal state h
- behavior mode variable m
- 2D foraging environment
- depleting food patch
- sparse risk signal
```

- trajectory visualization
- ablations for no-h and no-m

Constraints:

- keep dependencies minimal
- prefer numpy + matplotlib
- no heavy RL frameworks
- keep code small and inspectable

Before coding, summarize the implementation plan briefly, then proceed.

ローカル運用のおすすめはかなりはっきりしていて、

- VS Code で repo 編集
- Codex CLI で実装
- `tmux` でセッション保持
- GitHub に push
- 必要なら GitHub Actions でテスト

です。

今のテーマでは、**最初から自律度を上げすぎない**ほうがいいです。

まずは

1. scaffold 作成
2. 環境実装
3. agent 実装
4. 可視化
5. ablation

の順で区切って Codex CLI に投げるのが安全です。

必要なら次に、`minimal-enactive-agent` 用の **Codex CLI** 向け最初のコマンド例 と `tmux` 前提の夜間実行手順をまとめます。